EXP NO: 1 Date:	Downloading and installing Hadoop; Understanding different Hadoop modes. Startup scripts, Configuration files

Downloading and installing Hadoop:

AIM: To Install Apache Hadoop.

Hadoop software can be installed in three modes

Hadoop is a Java-based programming framework that supports the processing and storage of extremely large datasets on a cluster of inexpensive machines. It was the first major open source project in the big data playing field and is sponsored by the Apache Software Foundation.

Hadoop-2.7.3 is comprised of four main layers:

- Hadoop Common is the collection of utilities and libraries that support other Hadoop modules.
- HDFS, which stands for Hadoop Distributed File System, is responsible for persisting data to disk.
- YARN, short for Yet Another Resource Negotiator, is the "operating system" for HDFS.
- MapReduce is the original processing model for Hadoop clusters. It distributes work within the cluster or map, then organizes and reduces the results from the nodes into a response to a query. Many other processing models are available for the 2.x version of Hadoop.

Hadoop clusters are relatively complex to set up, so the project includes a stand-alone mode which is suitable for learning about Hadoop, performing simple operations, and debugging.

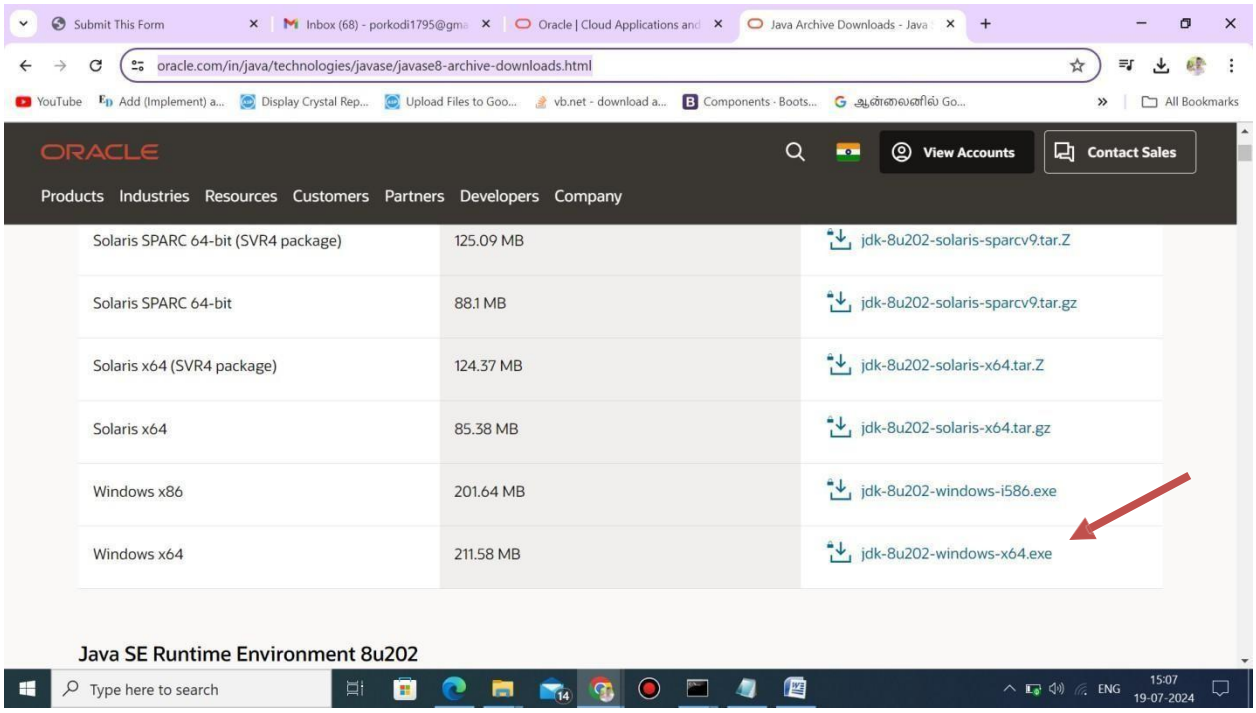
Procedure:

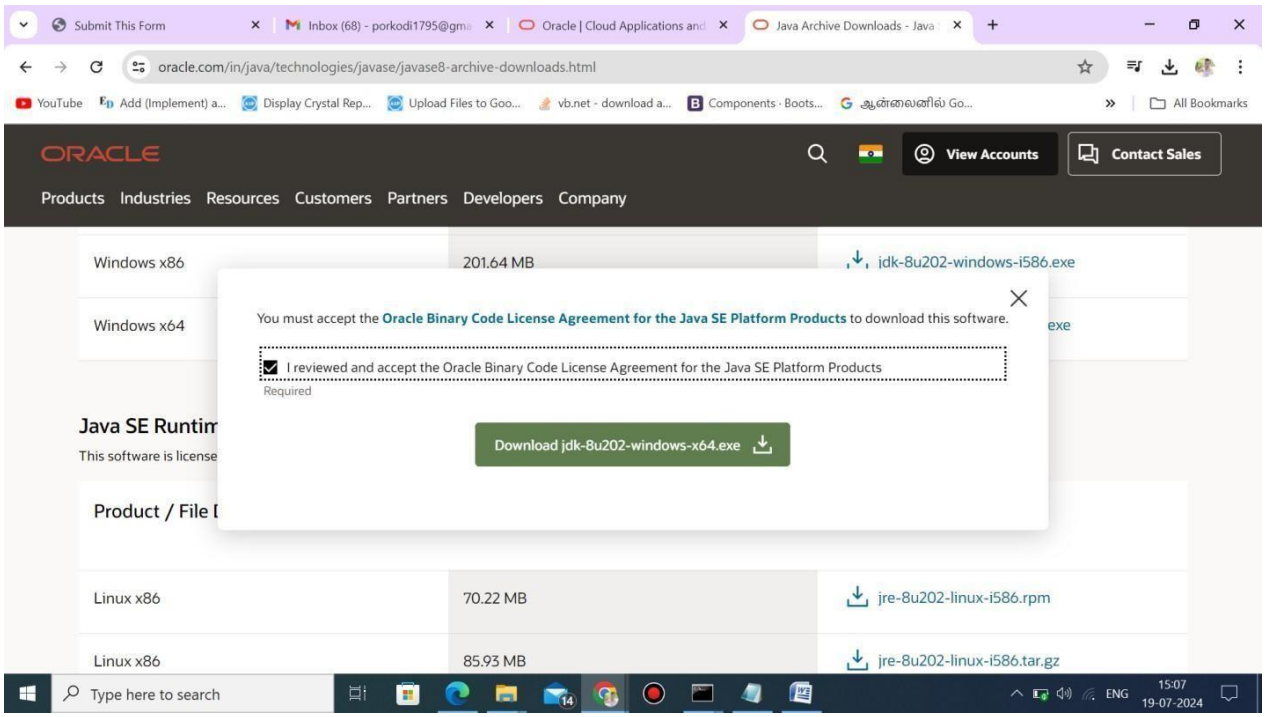
We will install Hadoop in stand-alone mode and run one of the example MapReduce programs it includes to verify the installation.

Prerequisites:

Step1: Installing Java 8 version.

Openjdk version "1.8.0_91" using the below link to login and download it.





Then click the button to register/signup to download .

https://www.oracle.com/in/java/technologies/javase/javase8-archive-downloads.html

OpenJDK Runtime Environment (build 1.8.0_91-8u91-b14-3ubuntu1~16.04.1-b14)

CCS334_BDA Lab

OpenJDK 64-Bit Server VM (build 25.91-b14, mixed mode)

This output verifies that OpenJDK has been successfully installed.

Step by step Hadoop 2.8.0 installation on Windows 10 Prepare:

These software's should be prepared to install Hadoop 2.8.0 on window 10 64 bits.

1) Download Hadoop 2.8.0

http://archive.apache.org/dist/hadoop/core//hadoop-2.8.0/hadoop2.8.0.tar.gz

2) Java JDK 1.8.0.zip

https://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads2133151.html

Note: To set the path for environment variables. i.e. JAVA_HOME

Step2: Installing Hadoop With Java in place, we'll visit the Apache Hadoop Releases page to find the most recent stable release. Follow the binary for the current release:

***Set up:**

1) Check either Java 1.8.0 is already installed on your system or not, use "Javac - version" to check Java version

2) If Java is not installed on your system then first install java under "C:\JAVA" Java setup

3) Extract files Hadoop 2.8.0.tar.gz or Hadoop-2.8.0.zip and place under "C:\Hadoop-2.8.0" hadoop

4) Set the path HADOOP_HOME Environment variable on windows 10(see Step 1, 2, 3 and 4 below) hadoop

5) Set the path JAVA_HOME Environment variable on windows 10(see Step 1, 2, 3 and 4 below) java

6) Next we set the Hadoop bin directory path and JAVA bin directory path

Configuration

a) File C:/Hadoop-2.8.0/etc/hadoop/core-site.xml, paste below xml paragraph and save this file.
<configuration>

<property>

<name>fs.defaultFS</name>

<value>hdfs://localhost:9000</value>

</property>

</configuration>

b) Rename "mapred-site.xml.template" to "mapred-site.xml" and edit this file C:/Hadoop-2.8.0/etc/hadoop/mapred-site.xml, paste below xml paragraph and save this file.

<configuration>

<property>

<name>mapreduce.framework.name</name>

<value>yarn</value>

</property>

</configuration>

c) Create folder "data" under "C:\Hadoop-2.8.0"

1) Create folder "datanode" under "C:\Hadoop-2.8.0\data"

2) Create folder "namenode" under "C:\Hadoop-2.8.0\data" data

d) Edit file C:\Hadoop-2.8.0/etc/hadoop/hdfs-site.xml, paste below xml paragraph and save this file.

<configuration>

<property>

<name>dfs.replication</name>

<value>1</value>

</property>

<property>

<name>dfs.namenode.name.dir</name>

<value>C:\hadoop-2.8.0\data\namenode</value>

</property>

<property>

<name>dfs.datanode.data.dir</name>

<value>C:\hadoop-2.8.0\data\datanode</value>

</property>

</configuration>

e) Edit file C:/Hadoop-2.8.0/etc/hadoop/yarn-site.xml, paste below xml paragraph

and save this file.

<configuration>

<property>

<name>yarn.nodemanager.aux-services</name>

<value>mapreduce_shuffle</value>

</property>

<property>

<name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</name>

<value>org.apache.hadoop.mapred.ShuffleHandler</value>

</property>

</configuration>

f) Edit file C:/Hadoop-2.8.0/etc/hadoop/hadoop-env.cmd by closing the command line "JAVA_HOME=%JAVA_HOME%" instead of set "JAVA_HOME=C:\Java" (On C:\java this is path to file jdk.18.0)

Hadoop Configuration

6) Download file Hadoop Configuration.zip https://github.com/MuhammadBilalYar/Hadoop-OnWindow/blob/master/Hadoop%20Configuration.zip)

7) Delete file bin on C:\Hadoop-2.8.0\bin, replaced by file bin on file just download (from Hadoop Configuration.zip).

8) Open cmd and typing command "hdfs namenode –format" .You will see hdfs namenode –format

Testing

9) Open cmd and change directory to "C:\Hadoop-2.8.0\sbin" and type "startall.cmd" to start apache.

10) Make sure these apps are running.

a) Name node

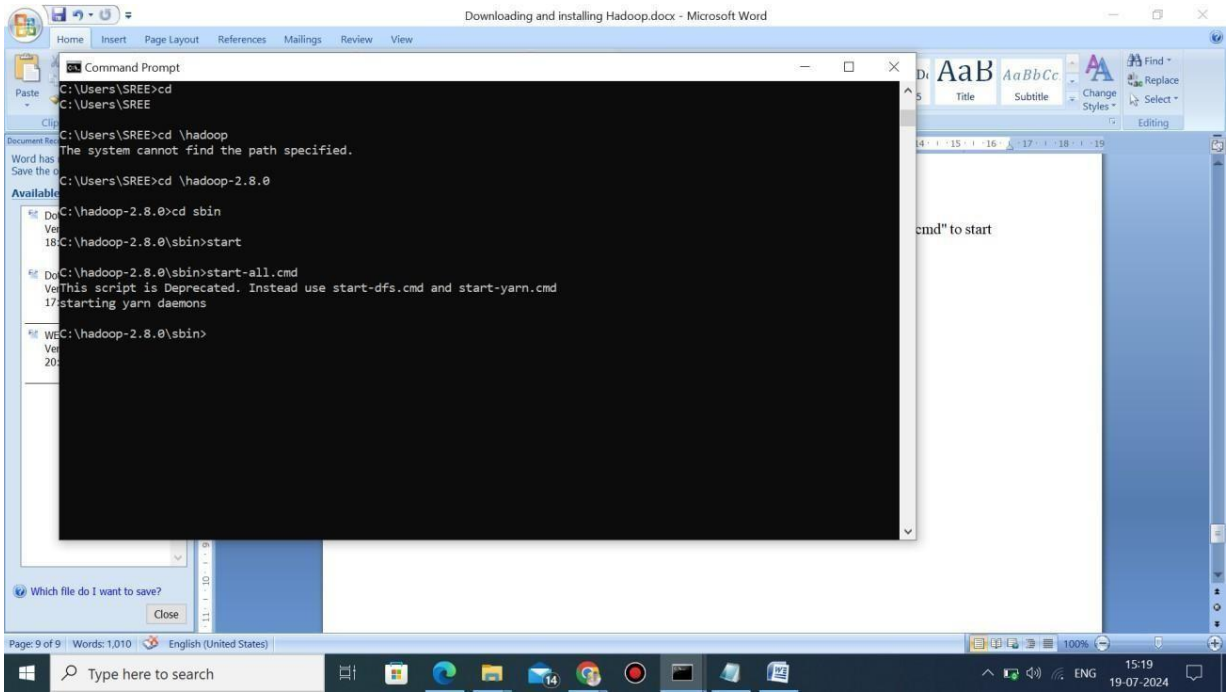
b) Hadoop data node

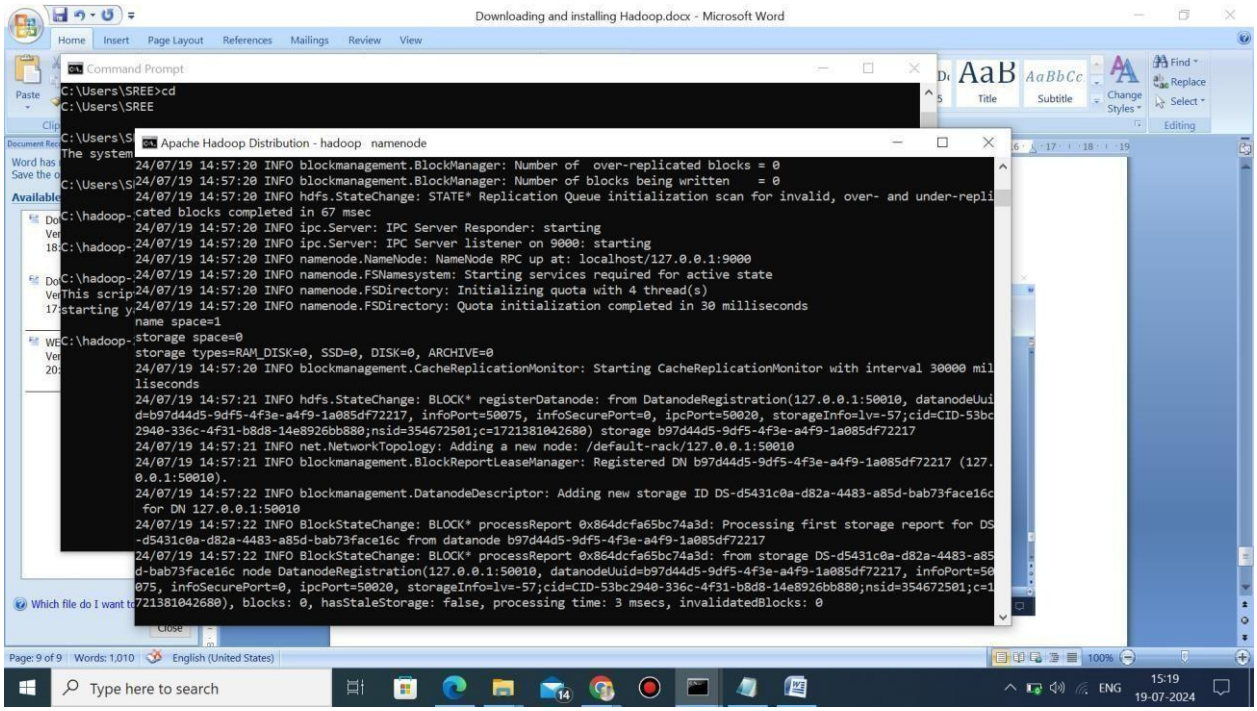
c) YARN Resource Manager

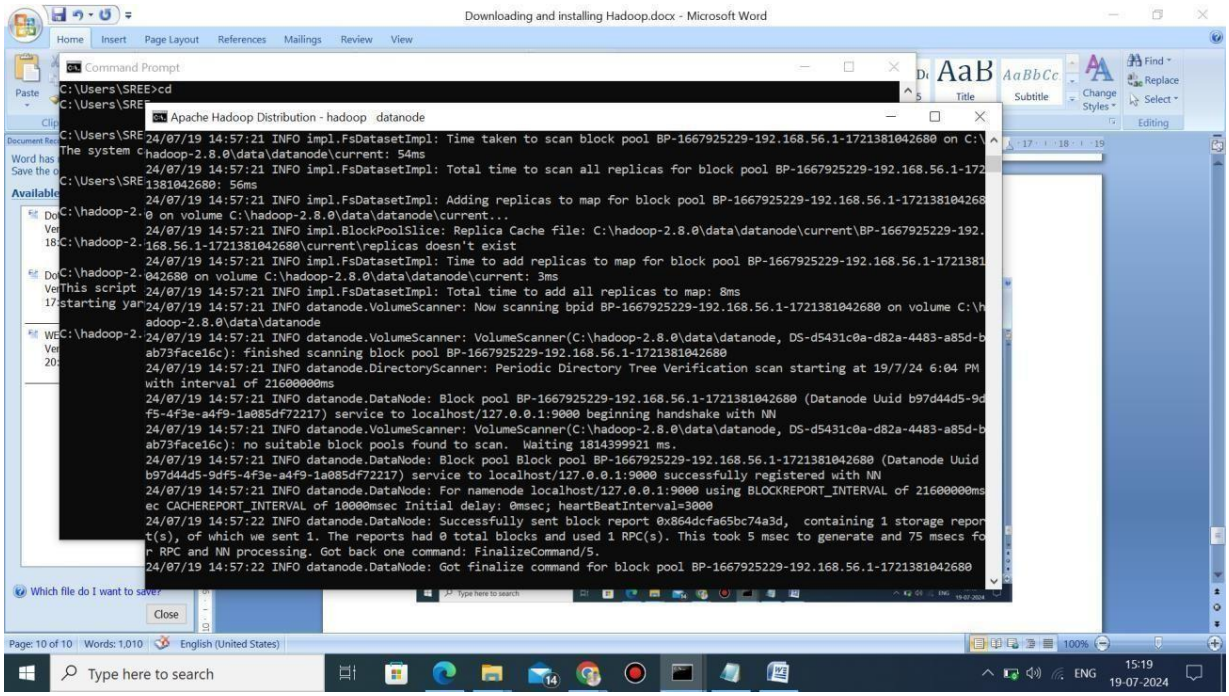
d) YARN Node Manager hadoop nodes

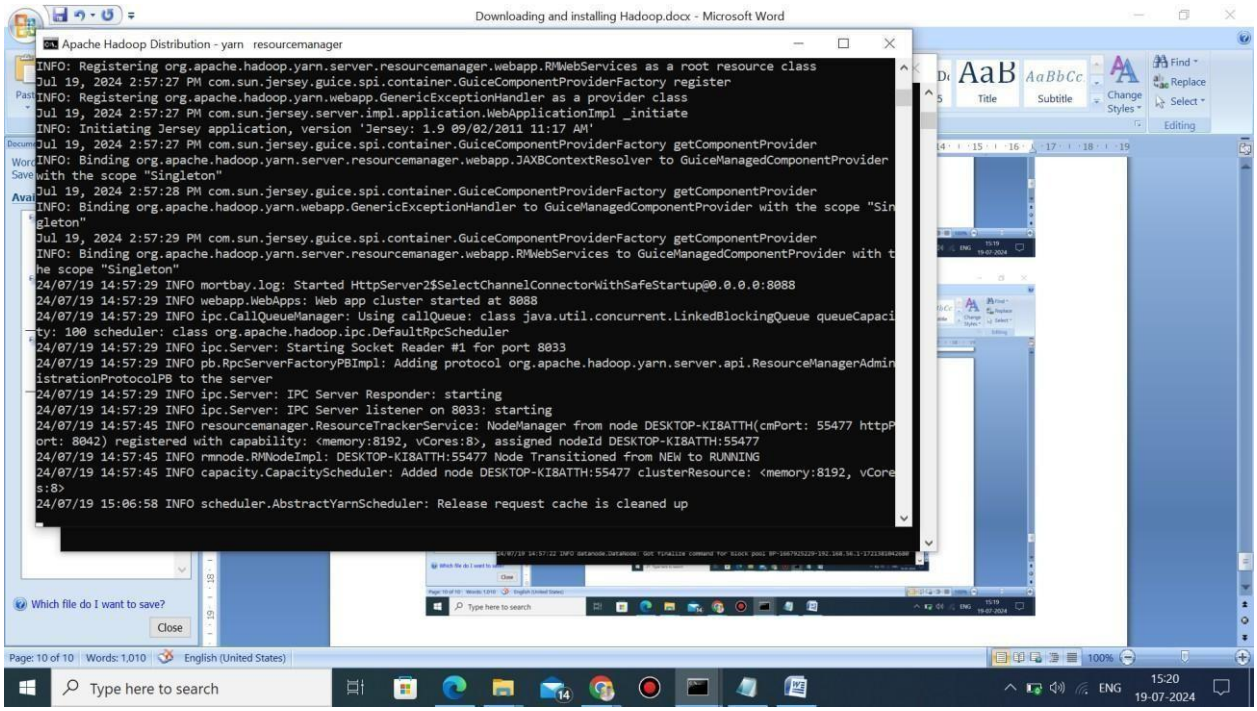
11) Open: http://localhost:8088

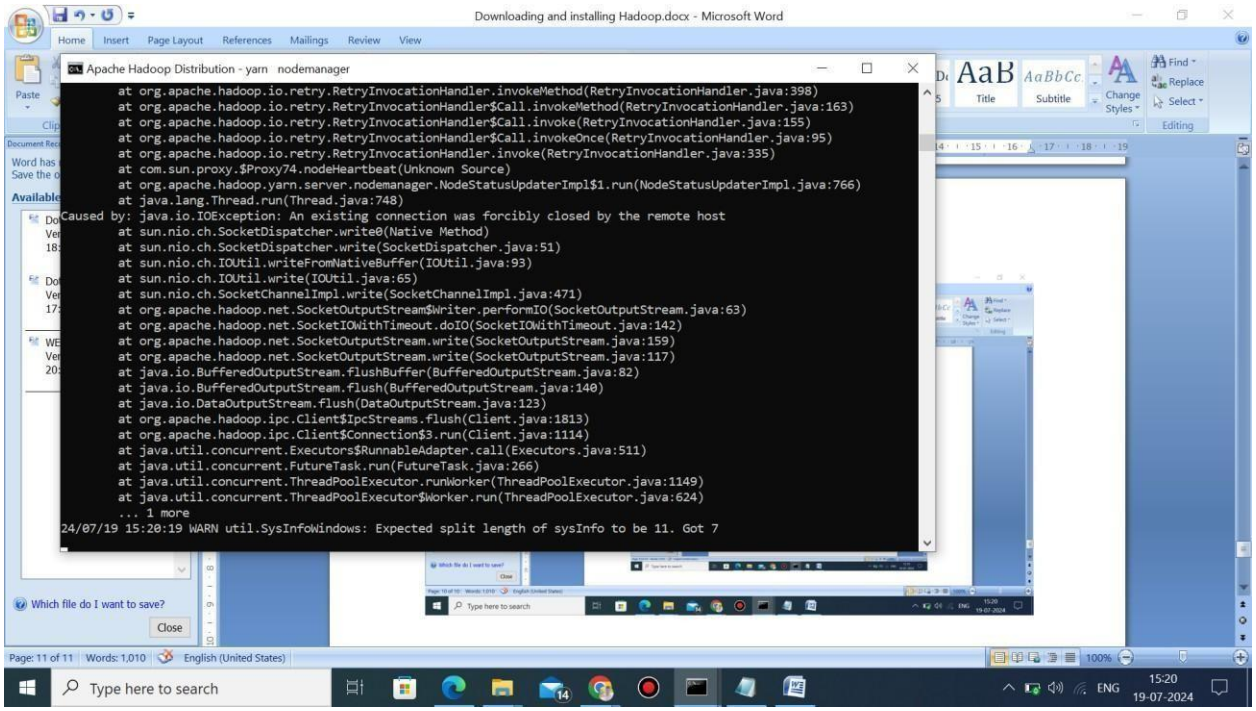
12) Open: http://localhost:50070




Downloading and installing Hadoop.docx - Microsoft Word
Command Prompt
C:\Users\SREE>cd
C:\Users\SREE
Apache Hadoop Distribution - hadoop namenode
24/07/19 14:57:20 INFO blockmanagement.BlockManager: Number of over-replicated blocks = 0
24/07/19 14:57:20 INFO blockmanagement.BlockManager: Number of blocks being written = 0
24/07/19 14:57:20 INFO hdfs.StateChange: STATE* Replication Queue initialization scan for invalid, over- and under-repli
cated blocks completed in 67 msec
24/07/19 14:57:20 INFO ipc.Server: IPC Server Responder: starting
24/07/19 14:57:20 INFO ipc.Server: IPC Server listener on 9000: starting
24/07/19 14:57:20 INFO namenode.NameNode: NameNode RPC up at: localhost/127.0.0.1:9000
24/07/19 14:57:20 INFO namenode.FSNamesystem: Starting services required for active state
24/07/19 14:57:20 INFO namenode.FSDirectory: Initializing quota with 4 thread(s)
24/07/19 14:57:20 INFO namenode.FSDirectory: Quota initialization completed in 30 milliseconds
name space=1
storage space=0
storage types=RAM_DISK=0, SSD=0, DISK=0, ARCHIVE=0
24/07/19 14:57:20 INFO blockmanagement.CacheReplicationMonitor: Starting CacheReplicationMonitor with interval 30000 mil
liseconds
24/07/19 14:57:21 INFO hdfs.StateChange: BLOCK* registerDatanode: from DatanodeRegistration(127.0.0.1:50010, datanodeUui
d=b97d44d5-9df5-4f3e-a4f9-1a085df72217, infoPort=50075, infoSecurePort=0, ipcPort=50020, storageInfo=lv=-57;cid=CID-53bc
2940-336c-4f31-b8d8-14e8926bb880;nsid=354672501;c=1721381042680) storage b97d44d5-9df5-4f3e-a4f9-1a085df72217
24/07/19 14:57:21 INFO net.NetworkTopology: Adding a new node: /default-rack/127.0.0.1:50010
24/07/19 14:57:21 INFO blockmanagement.BlockReportLeaseManager: Registered DN b97d44d5-9df5-4f3e-a4f9-1a085df72217 (127.
0.0.1:50010).
24/07/19 14:57:22 INFO blockmanagement.DatanodeDescriptor: Adding new storage ID DS-d5431c0a-d82a-4483-a85d-bab73face16c
for DN 127.0.0.1:50010
24/07/19 14:57:22 INFO BlockStateChange: BLOCK* processReport 0x864dcfa65bc74a3d: Processing first storage report for DS
-d5431c0a-d82a-4483-a85d-bab73face16c from datanode b97d44d5-9df5-4f3e-a4f9-1a085df72217
24/07/19 14:57:22 INFO BlockStateChange: BLOCK* processReport 0x864dcfa65bc74a3d: from storage DS-d5431c0a-d82a-4483-a85
d-bab73face16c node DatanodeRegistration(127.0.0.1:50010, datanodeUuid=b97d44d5-9df5-4f3e-a4f9-1a085df72217, infoPort=50
075, infoSecurePort=0, ipcPort=50020, storageInfo=lv=-57;cid=CID-53bc2940-336c-4f31-b8d8-14e8926bb880;nsid=354672501;c=1
721381042680), blocks: 0, hasStaleStorage: false, processing time: 3 msecs, invalidatedBlocks: 0
Page: 9 of 9 Words: 1,010 English (United States)
Type here to search
ENG 15:19 19-07-2024


Downloading and installing Hadoop.docx - Microsoft Word
Command Prompt
C:\Users\SREE>cd
Apache Hadoop Distribution - hadoop datanode
24/07/19 14:57:21 INFO impl.FsDatasetImpl: Time taken to scan block pool BP-1667925229-192.168.56.1-1721381042680 on C:\
hadoop-2.8.0\data\datanode\current: 54ms
24/07/19 14:57:21 INFO impl.FsDatasetImpl: Total time to scan all replicas for block pool BP-1667925229-192.168.56.1-172
1381042680: 56ms
24/07/19 14:57:21 INFO impl.FsDatasetImpl: Adding replicas to map for block pool BP-1667925229-192.168.56.1-172138104268
0 on volume C:\hadoop-2.8.0\data\datanode\current...
24/07/19 14:57:21 INFO impl.BlockPoolSlice: Replica Cache file: C:\hadoop-2.8.0\data\datanode\current\BP-1667925229-192.
168.56.1-1721381042680\current\replicas doesn't exist
24/07/19 14:57:21 INFO impl.FsDatasetImpl: Time to add replicas to map for block pool BP-1667925229-192.168.56.1-1721381
042680 on volume C:\hadoop-2.8.0\data\datanode\current: 3ms
24/07/19 14:57:21 INFO impl.FsDatasetImpl: Total time to add all replicas to map: 8ms
24/07/19 14:57:21 INFO datanode.VolumeScanner: Now scanning bpid BP-1667925229-192.168.56.1-1721381042680 on volume C:\h
adoop-2.8.0\data\datanode
24/07/19 14:57:21 INFO datanode.VolumeScanner: VolumeScanner(C:\hadoop-2.8.0\data\datanode, DS-d5431c0a-d82a-4483-a85d-b
ab73face16c): finished scanning block pool BP-1667925229-192.168.56.1-1721381042680
24/07/19 14:57:21 INFO datanode.DirectoryScanner: Periodic Directory Tree Verification scan starting at 19/7/24 6:04 PM
with interval of 21600000ms
24/07/19 14:57:21 INFO datanode.DataNode: Block pool BP-1667925229-192.168.56.1-1721381042680 (Datanode Uuid b97d44d5-9d
f5-4f3e-a4f9-1a085df72217) service to localhost/127.0.0.1:9000 beginning handshake with NN
24/07/19 14:57:21 INFO datanode.VolumeScanner: VolumeScanner(C:\hadoop-2.8.0\data\datanode, DS-d5431c0a-d82a-4483-a85d-b
ab73face16c): no suitable block pools found to scan. Waiting 1814399921 ms.
24/07/19 14:57:21 INFO datanode.DataNode: Block pool Block pool BP-1667925229-192.168.56.1-1721381042680 (Datanode Uuid
b97d44d5-9df5-4f3e-a4f9-1a085df72217) service to localhost/127.0.0.1:9000 successfully registered with NN
24/07/19 14:57:21 INFO datanode.DataNode: For namenode localhost/127.0.0.1:9000 using BLOCKREPORT_INTERVAL of 21600000ms
ec CACHEREPORT_INTERVAL of 10000msec Initial delay: 0msec; heartBeatInterval=3000
24/07/19 14:57:22 INFO datanode.DataNode: Successfully sent block report 0x864dcfa65bc74a3d, containing 1 storage repor
t(s), of which we sent 1. The reports had 0 total blocks and used 1 RPC(s). This took 5 msec to generate and 75 msecs fo
r RPC and NN processing. Got back one command: FinalizeCommand/5.
24/07/19 14:57:22 INFO datanode.DataNode: Got finalize command for block pool BP-1667925229-192.168.56.1-1721381042680
Page: 10 of 10 Words: 1,010 English (United States)
Type here to search
ENG 15:19 19-07-2024





Understanding Different Hadoop Modes

Hadoop can operate in three different modes:

1. **Standalone Mode (Local Mode):**

- It is Default mode.
- No configuration needed.
- Suitable for debugging and development.
- All Hadoop components run in a single Java process.
- Data is stored in the local file system.

2. Pseudo-Distributed Mode:

- Simulates a fully distributed cluster on a single machine.
- Each Hadoop component runs as a separate Java process.
- Requires minimal configuration changes.
- Suitable for testing and development.

3. Fully Distributed Mode:

- Hadoop runs on multiple machines (nodes).
- Suitable for production.
- Requires extensive configuration.
- Data is stored in HDFS (Hadoop Distributed File System).

Startup Scripts and Configuration Files

Key Configuration Files

1. hadoop-env.sh: Sets environment variables required by Hadoop. Located in $HADOOP_HOME/etc/hadoop.

2. core-site.xml: Contains configuration settings for Hadoop Core, including HDFS settings.

3. hdfs-site.xml: Contains configuration settings for HDFS.

4. mapred-site.xml: Contains configuration settings for MapReduce.

5. yarn-site.xml: Contains configuration settings for YARN.

Configuration for Pseudo-Distributed Mode

Edit the configuration code for given below

core-site.xml(C:\hadoop-2.8.0\etc\hadoop)

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
```

```
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
```

hdfs-site.xml:

```
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>file:///usr/local/hadoop/hdfs/namenode</value>
    </property>
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>file:///usr/local/hadoop/hdfs/datanode</value>
    </property>
</configuration>
```

mapred-site.xml:

```
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>
```

yarn-site.xml:

```
<configuration>
    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>
</configuration>
```

Starting Hadoop

1. **Format the NameNode (first time only)(Type the below command in cmd)**

hdfs namenode -format

2. **Start HDFS: (Type the below command in cmd)**

start-dfs.sh

3. **Start YARN:**

start-yarn.sh

Verify the Setup:

HDFS web UI: http://localhost:9870

YARN ResourceManager web UI: http://localhost:8088

Cmd Commands:

C:\Users\SREE>javac

Shows java installed or not and version

C:\Users\SREE>java -version

C:\Users\SREE>hdfs namenode –format

Hadoop configured or not

C:\Users\SREE>cd

C:\Users\SREE

C:\Users\SREE>cd \hadoop

The system cannot find the path specified.

C:\Users\SREE>cd \hadoop-2.8.0

C:\hadoop-2.8.0>cd sbin

C:\hadoop-2.8.0\sbin>start

C:\hadoop-2.8.0\sbin>start-all.cmd

This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd

starting yarn daemon

```
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>C:\hadoop-2.8.0\data\namenode</value>
    </property>
    <property>
```

```
        <name>dfs.datanode.data.dir</name>
        <value>C:\hadoop-2.8.0\data\datanode</value>
    </property>
</configuration>
```

RESULT:

EXP NO: 2	Hadoop Implementation of file management tasks, such as Adding files and directories, retrieving files and Deleting files
Date:	

AIM:

To run Hadoop's file management tasks can be performed using the Hadoop Distributed File System (HDFS) commands. These tasks include adding files and directories, retrieving files, and deleting files.

To run the command:

C:\Users\SREE>cd

C:\Users\SREE

C:\Users\SREE>cd \hadoop

The system cannot find the path specified.

C:\Users\SREE>cd \hadoop-2.8.0

C:\hadoop-2.8.0>cd sbin

C:\hadoop-2.8.0\sbin>start

C:\hadoop-2.8.0\sbin>start-all.cmd

This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd

starting yarn daemons

C:\hadoop-2.8.0\sbin>start-dfs.cmd

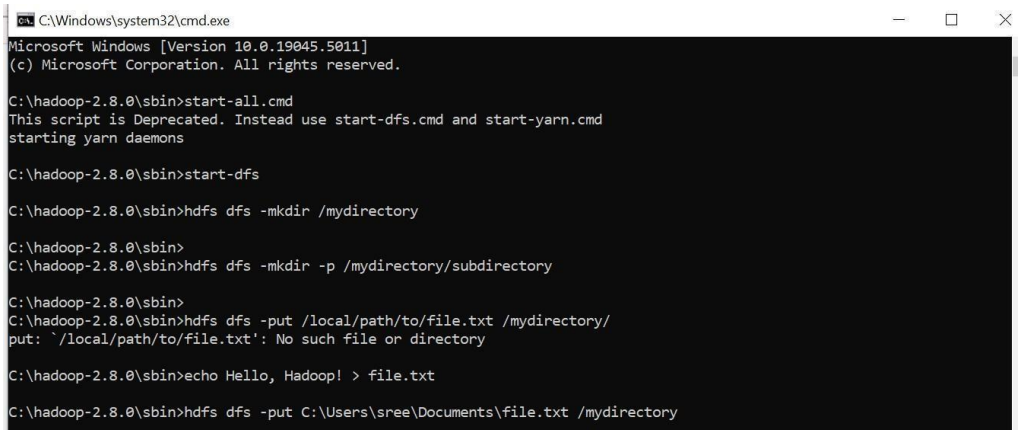
C:\hadoop-2.8.0\sbin>

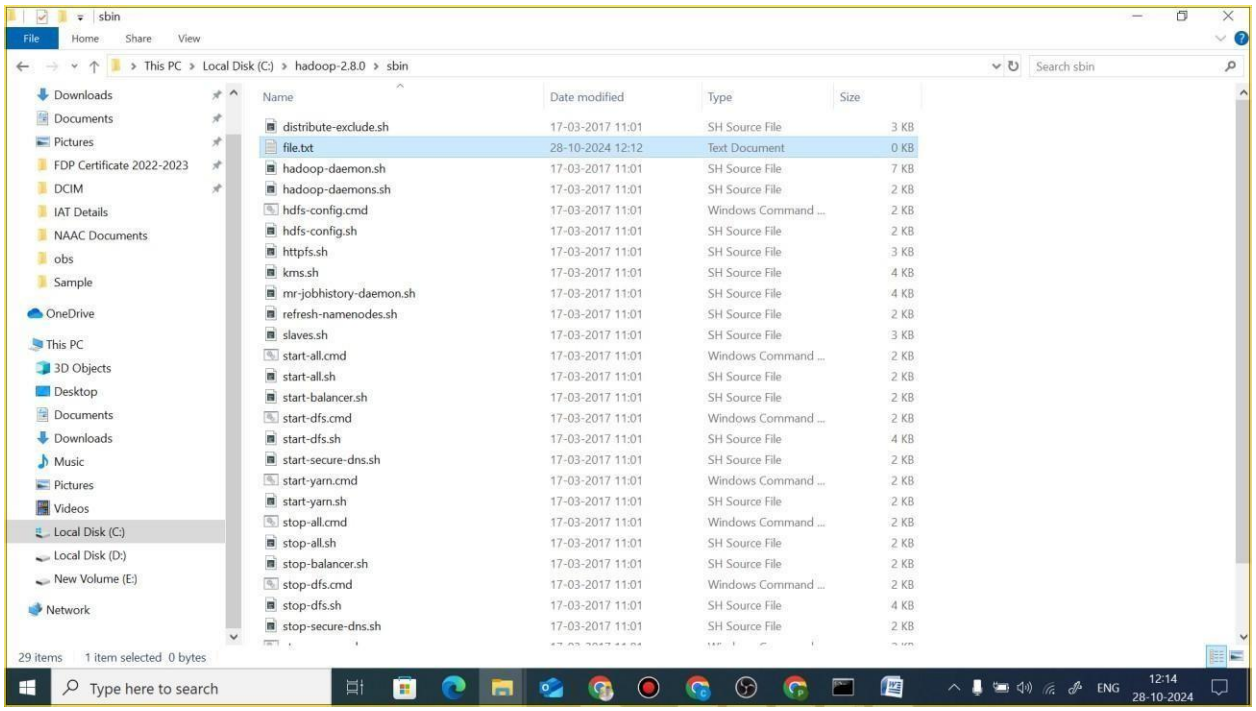
File Management Tasks in Hadoop

1. Adding Files and Directories

Creating a Directory:

To create a directory in HDFS, use the mkdir command.





Automatically create the following folder with the content.



The command hdfs dfs -put /home/user/Documents/file.txt /mydirectory/ is used to upload a file from your local filesystem to the Hadoop Distributed File System (HDFS).

hdfs:

This indicates that you are using the Hadoop FileSystem shell commands.

dfs:

This specifies that you are using commands related to the Hadoop Distributed File System.

-put:

This is the command that uploads a file or directory from the local filesystem to HDFS. It copies the file from your local system into HDFS.

/home/user/Documents/file.txt:

This is the source path, specifying the location of the file on your local filesystem that you want to upload. In this case, it's a text file named file.txt located in the Documents directory of the user's home directory.

/mydirectory/:

This is the destination path in HDFS where the file will be uploaded. In this case, it's a directory named mydirectory in HDFS.

Purpose of the Command

File Upload: The primary purpose is to transfer a file from the local filesystem to HDFS, making it accessible for processing by Hadoop applications and frameworks (like MapReduce, Hive, etc.).

Data Management: Uploading files to HDFS allows you to manage large datasets across a distributed environment, enabling scalability and redundancy.

Integration with Hadoop Ecosystem: By placing files in HDFS, you can utilize the full power of the Hadoop ecosystem, which is designed for big data processing.

PROCEDURE and STEPS

Step 1: Set Up Hadoop

Open Command Prompt: Press Win + R, type cmd, and hit Enter.

Set Environment Variables: Ensure that the Hadoop bin directory is in your system's PATH. You can set this in the command prompt temporarily or configure it permanently via System Properties.

set PATH=%PATH%;C:\hadoop-2.8.0\bin

Start Hadoop Services: If you have a pseudo-distributed setup, navigate to your Hadoop installation directory and run the following commands:

cd C:\hadoop-2.8.0\sbin

start-dfs.cmd

start-yarn.cmd

Step 2: Adding Files to HDFS

Create a Directory in HDFS: Before adding files, you may want to create a directory.

hadoop fs -mkdir /user/yourusername

hadoop fs -mkdir /user/sree (inside the c folder goto usethe directory)

Add Files to HDFS: Use the following command to upload a file from your local filesystem to HDFS.

hadoop fs -put C:\path\to\your\file.txt /user/yourusername/

hadoop fs -put C:\path\to\your\file.txt /user/sree/

Step 3: Retrieving Files from HDFS

Copy Files from HDFS to Local Filesystem: To download a file from HDFS back to your local filesystem, use:

hadoop fs -get /user/yourusername/file.txt C:\path\to\local\directory\

hadoop fs -get /user/sree/file.txt C:\path\to\local\directory\

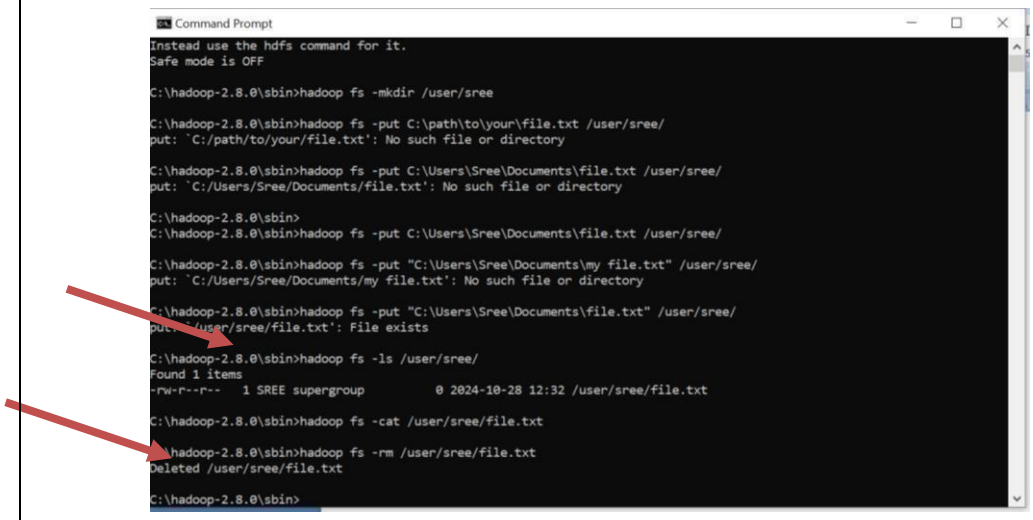
Step 4: Deleting Files in HDFS

Delete a File from HDFS: If you need to delete a specific file, use the following command:

hadoop fs -rm /user/yourusername/file.txt

hadoop fs -rm /user/sree/file.txt

OUTPUT



Delete a Directory in HDFS: To remove a directory (and its contents), use:

hadoop fs -rm -r /user/yourusername/directory_name

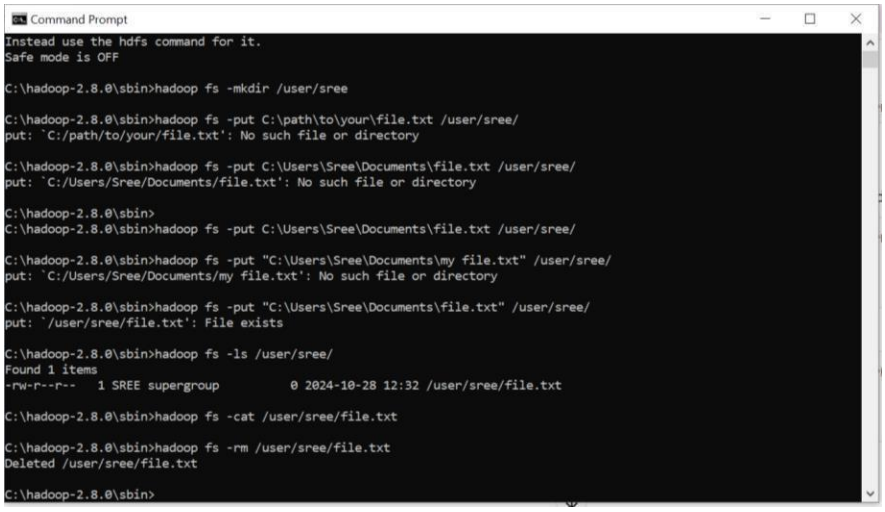
Step 5: Verify Operations

To list files in a directory:

hadoop fs -ls /user/yourusername/

To check the contents of a file in HDFS:

hadoop fs -cat /user/yourusername/file.txt



RESULT:

EXP NO: 3	Implement of Matrix Multiplication with Hadoop Map Reduce
Date:	

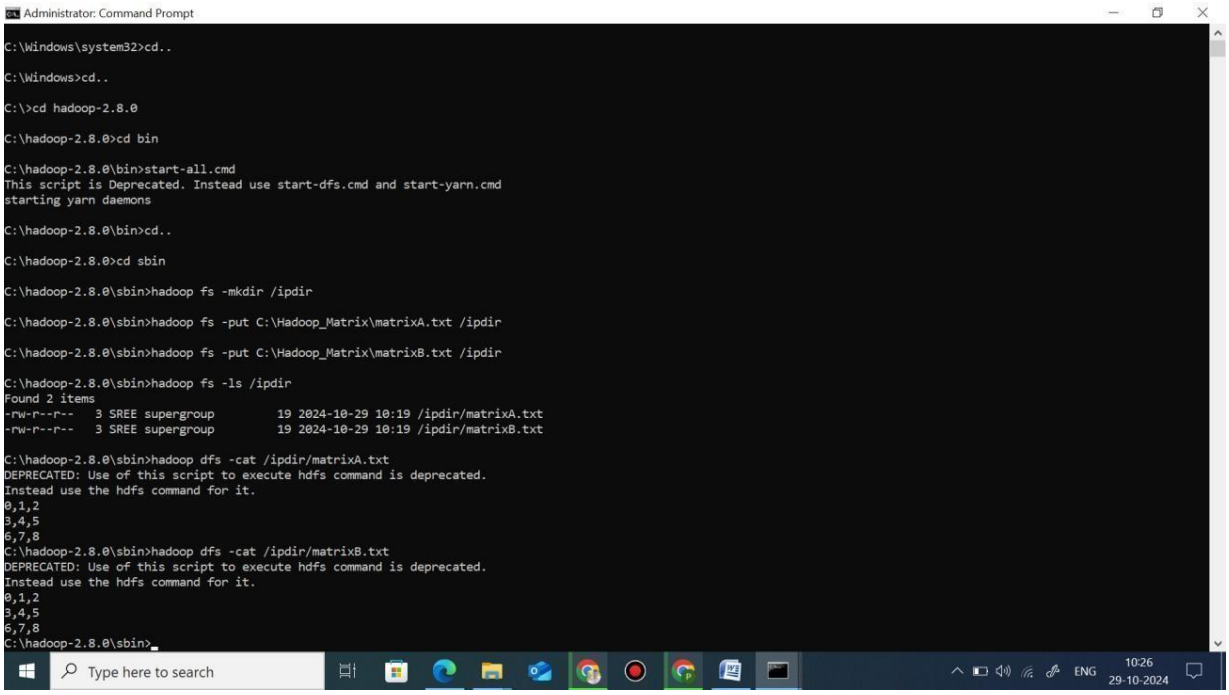
Aim :

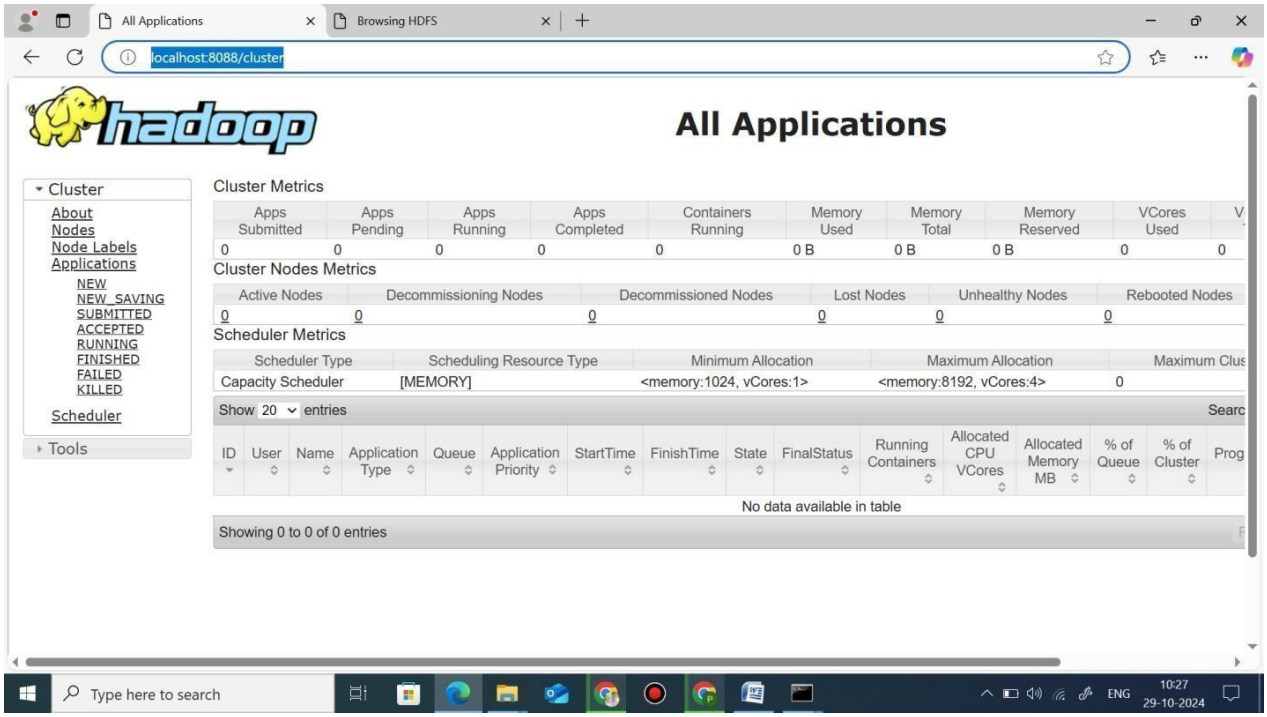
To implement matrix multiplication using the MapReduce paradigm in Hadoop.

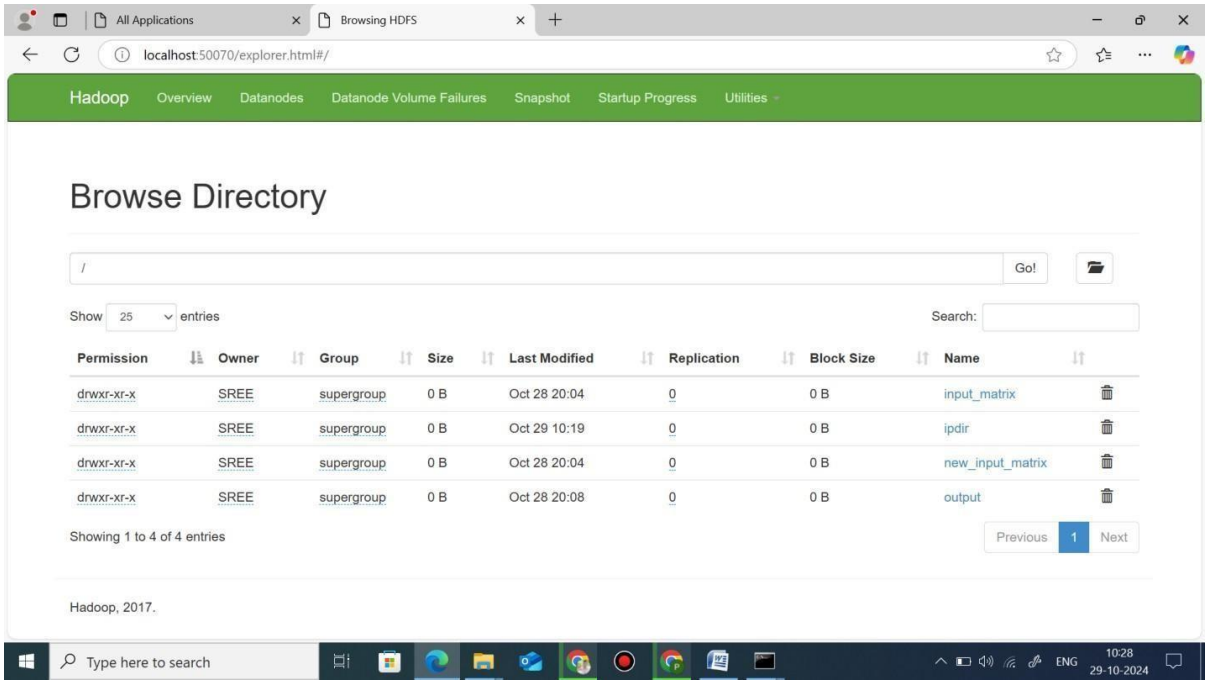
We'll define the matrices, write the MapReduce code, compile it, and then run the job to see the output.

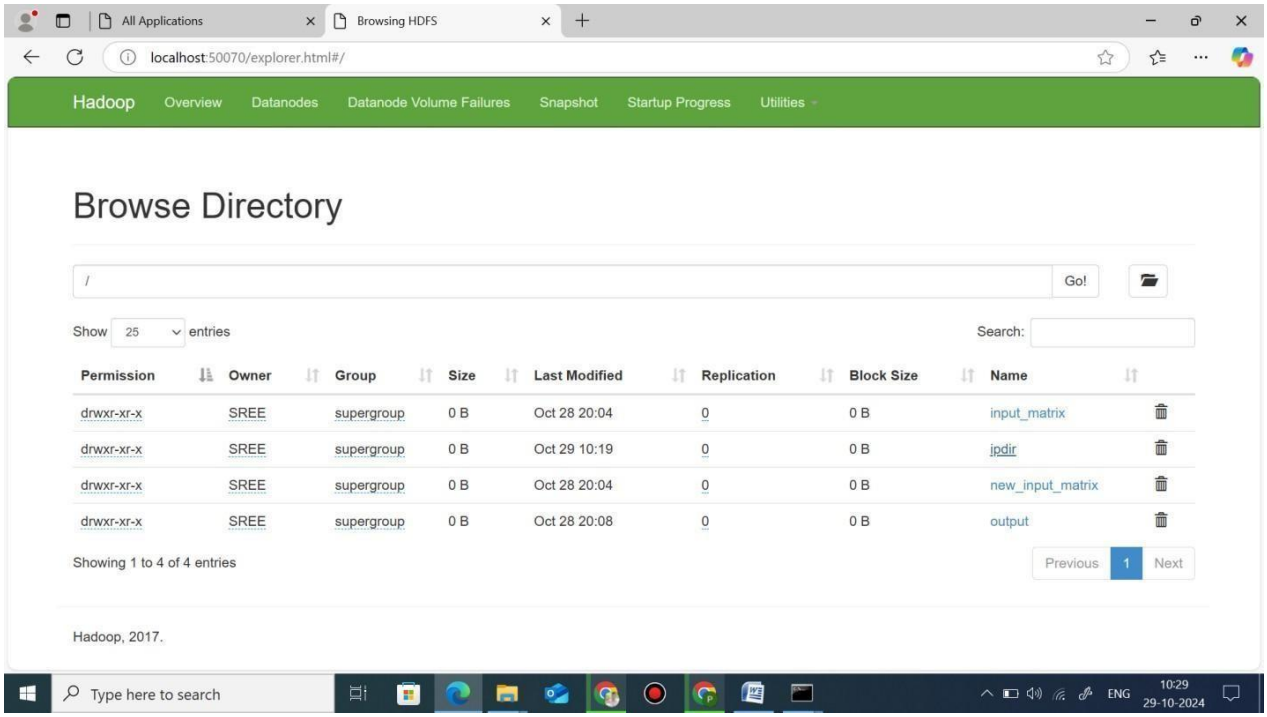
Requirements:

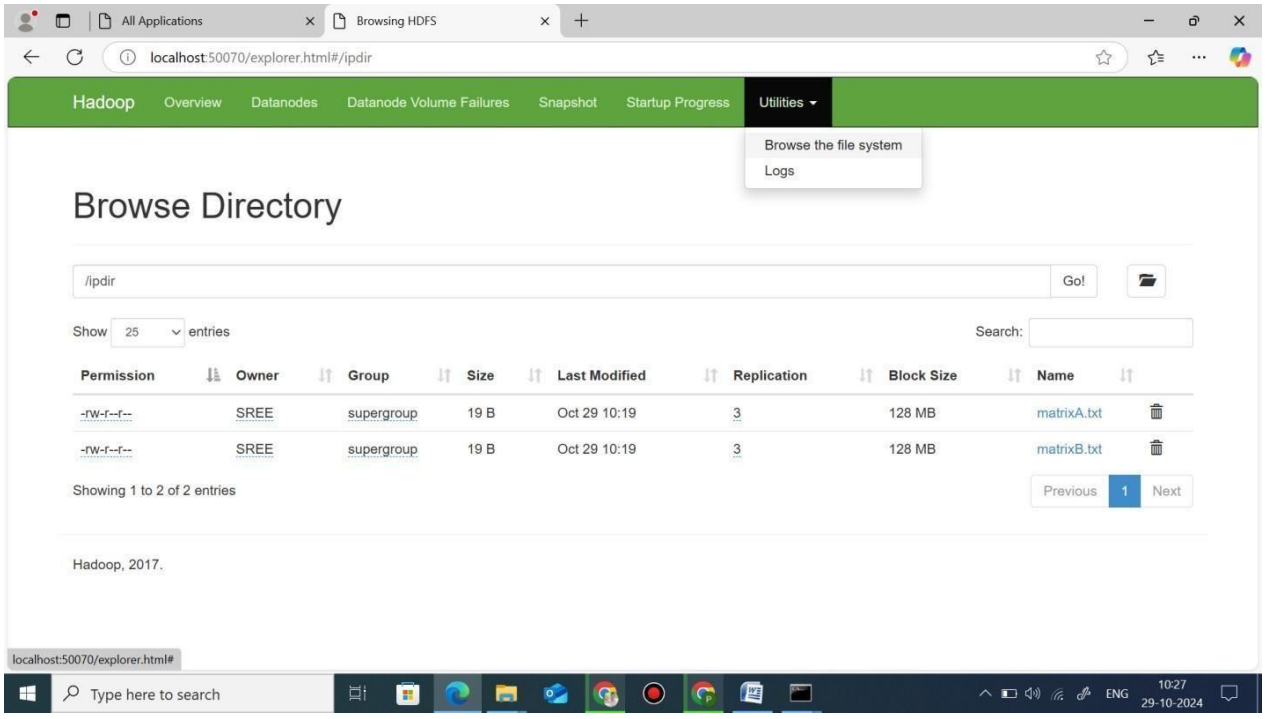
- Java Development Kit (JDK) installed and configured.
- Hadoop installed and properly configured on your Windows machine.
- Basic knowledge of Java and Hadoop concepts.




All Applications
Browsing HDFS
localhost:8088/cluster
hadoop
All Applications
Cluster
About
Nodes
Node Labels
Applications
NEW
NEW_SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
Scheduler
Tools
Cluster Metrics
Apps Submitted
Apps Pending
Apps Running
Apps Completed
Containers Running
Memory Used
Memory Total
Memory Reserved
VCores Used
0 0 0 0 0 0 B 0 B 0 B 0 0
Cluster Nodes Metrics
Active Nodes
Decommissioning Nodes
Decommissioned Nodes
Lost Nodes
Unhealthy Nodes
Rebooted Nodes
0 0 0 0 0 0
Scheduler Metrics
Scheduler Type
Scheduling Resource Type
Minimum Allocation
Maximum Allocation
Capacity Scheduler
[MEMORY]
<memory:1024, vCores:1>
<memory:8192, vCores:4>
0
Show 20 entries
ID
User
Name
Application Type
Queue
Application Priority
StartTime
FinishTime
State
FinalStatus
Running Containers
Allocated CPU VCores
Allocated Memory MB
% of Queue
% of Cluster
No data available in table
Showing 0 to 0 of 0 entries
Type here to search
ENG
10:27
29-10-2024


All Applications
Browsing HDFS
localhost:50070/explorer.html#/
Hadoop
Overview
Datanodes
Datanode Volume Failures
Snapshot
Startup Progress
Utilities
Browse Directory
/
Go!
Show 25 entries
Search:
Permission
Owner
Group
Size
Last Modified
Replication
Block Size
Name
drwxr-xr-x SREE supergroup 0 B Oct 28 20:04 0 0 B input_matrix
drwxr-xr-x SREE supergroup 0 B Oct 29 10:19 0 0 B ipdir
drwxr-xr-x SREE supergroup 0 B Oct 28 20:04 0 0 B new_input_matrix
drwxr-xr-x SREE supergroup 0 B Oct 28 20:08 0 0 B output
Showing 1 to 4 of 4 entries
Previous
1
Next
Hadoop, 2017.
Type here to search
ENG
10:28
29-10-2024





Map.java

```
package com.mapreduce.wc;


import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
//import org.apache.hadoop.mapreduce.Mapper;
```

```
import java.io.IOException;
public class Map extends org.apache.hadoop.mapreduce.Mapper<LongWritable, Text, Text,
Text>
{
	@Override
public void map(LongWritable key, Text value, Context context)
throws IOException, InterruptedException {
		Configuration conf = context.getConfiguration();
		int m = Integer.parseInt(conf.get("m"));
		int p = Integer.parseInt(conf.get("p"));
String line = value.toString();
// (M, i, j, Mij);
String[] indicesAndValue = line.split(",");
Text outputKey = new Text();
Text outputValue = new Text();
if (indicesAndValue[0].equals("M")) {
for (int k = 0; k < p; k++)
{ outputKey.set(indicesAndValue[1] + "," + k);
// outputKey.set(i,k);
outputValue.set(indicesAndValue[0] + "," + indicesAndValue[2]
+ "," + indicesAndValue[3]);
// outputValue.set(M,j,Mij);
context.write(outputKey, outputValue);
}
} else {
	// (N, j, k, Njk);
	for (int i = 0; i < m; i++) {
	outputKey.set(i + "," + indicesAndValue[2]); outputValue.set("N," +
indicesAndValue[1] + ","
	+ indicesAndValue[3]); context.write(outputKey, outputValue);
	}
	}
	}
}
```

MatrixMultiply.java

```
package com.mapreduce.wc;

import org.apache.hadoop.conf.*;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
public class MatrixMultiply {
public static void main(String[] args) throws Exception { if (args.length != 2)
{ System.err.println("Usage: MatrixMultiply <in_dir> <out_dir>");
System.exit(2);
}
Configuration conf = new Configuration();
conf.set("m", "1000");
conf.set("n", "100");
conf.set("p", "1000");
```

```
@SuppressWarnings("deprecation")
Job job = new Job(conf, "MatrixMultiply");
job.setJarByClass(MatrixMultiply.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);
job.setMapperClass(Map.class);
job.setReducerClass(Reduce.class);
job.setInputFormatClass(TextInputFormat.class);
job.setOutputFormatClass(TextOutputFormat.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
job.waitForCompletion(true);
}
}
```

Reduce.java

```
package com.mapreduce.wc;


import org.apache.hadoop.io.Text;
// import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;
import java.util.HashMap;
public class Reduce
extends org.apache.hadoop.mapreduce.Reducer<Text, Text, Text, Text> { @Override
public void reduce(Text key, Iterable<Text> values, Context context)
throws IOException, InterruptedException
{ String[] value;
//key=(i,k),
//Values = [(M/N,j,V/W),..]
HashMap<Integer, Float> hashA = new HashMap<Integer, Float>(); HashMap<Integer,
Float> hashB = new HashMap<Integer, Float>(); for (Text val : values) {
value = val.toString().split(",");
if (value[0].equals("M")) {
hashA.put(Integer.parseInt(value[1]), Float.parseFloat(value[2])); } else
{ hashB.put(Integer.parseInt(value[1]), Float.parseFloat(value[2]));
}
}
int n = Integer.parseInt(context.getConfiguration().get("n"));
float result = 0.0f;
float m_ij;
float n_jk;
for (int j = 0; j < n; j++) {
m_ij = hashA.containsKey(j) ? hashA.get(j) : 0.0f; n_jk = hashB.containsKey(j) ?
hashB.get(j) : 0.0f; result += m_ij * n_jk;
}
if (result != 0.0f)
{ context.write(null,
new Text(key.toString() + "," + Float.toString(result)));
}
}
}
```

Procedure

Step 1: Define the Matrices

Let's assume we have two matrices:

Matrix A (2x3):

1 2 3

4 5 6

Matrix B (3x2):

7 8

9 10

11 12

The result of multiplying Matrix A and Matrix B will be a new matrix C (2x2):

C[0][0] = 1*7 + 2*9 + 3*11 = 58

C[0][1] = 1*8 + 2*10 + 3*12 = 64

C[1][0] = 4*7 + 5*9 + 6*11 = 139

C[1][1] = 4*8 + 5*10 + 6*12 = 154

Step 2: Create Input Files

Create two input files for the matrices.

Matrix A (input_a.txt)

1,2,3

4,5,6

Matrix B (input_b.txt)

7,8

9,10

11,12

Step 3: Write the MapReduce Code

Create a Java file named MatrixMultiplication.java.

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class MatrixMultiplication {
    public static class MapperA extends Mapper<Object, Text, Text, Text> {
        public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
            String[] matrixA = value.toString().split(",");
            for (int j = 0; j < matrixA.length; j++) {
                context.write(new Text("A" + key), new Text(j + "," + matrixA[j]));
            }
        }
    }

    public static class MapperB extends Mapper<Object, Text, Text, Text> {
        public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
```

```
            String[] matrixB = value.toString().split(",");
            for (int i = 0; i < matrixB.length; i++) {
                context.write(new Text(i + "," + key), new Text(matrixB[i]));
            }
        }
    }
    public static class ReducerMultiply extends Reducer<Text, Text, Text, Text> {
        public void reduce(Text key, Iterable<Text> values, Context context) throws IOException,
InterruptedException {
            int sum = 0;
            for (Text val : values) {
                String[] parts = val.toString().split(",");
                sum += Integer.parseInt(parts[1]) * Integer.parseInt(parts[0]);
            }
            context.write(key, new Text(String.valueOf(sum)));
        }
    }
    public static void main(String[] args) throws Exception
        { Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Matrix Multiplication");
        job.setJarByClass(MatrixMultiplication.class);
        job.setMapperClass(MapperA.class);
        job.setMapperClass(MapperB.class);
        job.setReducerClass(ReducerMultiply.class);
                job.setOutputKeyClass(Text.class);
```

```
        job.setOutputValueClass(Text.class);
            FileInputFormat.addInputPath(job, new Path(args[0]));
        FileInputFormat.addInputPath(job, new Path(args[1]));
        FileOutputFormat.setOutputPath(job, new Path(args[2]));
            System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Step 4: Compile the Code

Open the command prompt and navigate to the directory containing your Java file.

Compile the Java file:

"C:\Java\bin\javac" -classpath "C:\hadoop-2.8.0\lib*" -d C:\Hadoop_Matrix\output\classes C:\Hadoop_Matrix\MatrixMultiplication.java

Step 5: Create the JAR File

After compilation, create a JAR file:

"C:\Java\bin\jar" cf C:\Hadoop_Matrix\output\classes\matrix-multiplication.jar -C C:\Hadoop_Matrix\output\classes .

Step 6: Prepare Input in HDFS

Assuming your Hadoop is running, load the input files into HDFS.

hadoop fs -mkdir /input_matrix

hadoop fs -put C:\Hadoop_Matrix\input_a.txt /input_matrix/

hadoop fs -put C:\Hadoop_Matrix\input_b.txt /input_matrix/

Step 7: Run the MapReduce Job

Run the MapReduce job with:

hadoop jar C:\Hadoop_Matrix\output\classes\matrix-multiplication.jar MatrixMultiplication /input_matrix/input_a.txt /input_matrix/input_b.txt /output_matrix

Step 8: View the Output

Check the output:

hadoop fs -ls /output_matrix

hadoop fs -cat /output_matrix/part-r-00000

Expected Output

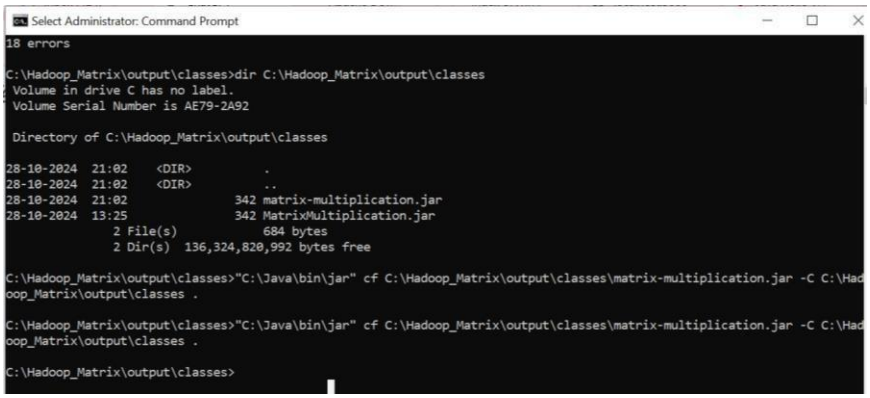
The output should display the results of the matrix multiplication:

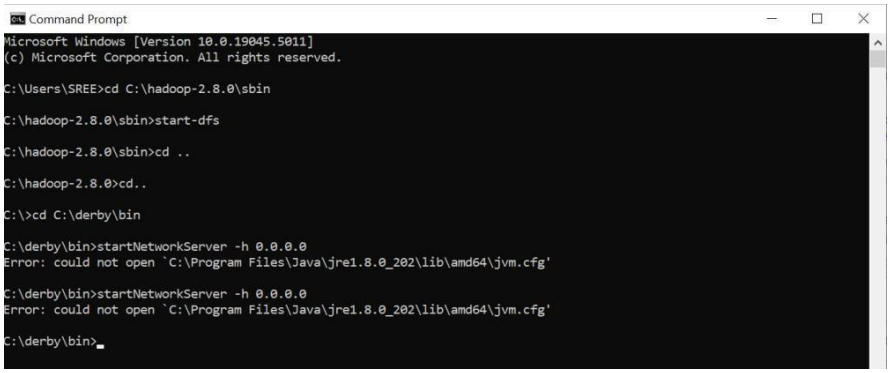
A0 58

A1 64

A0 139

A1 154





import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

```
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MatrixMultiplication {

    public static void main(String[] args) throws Exception

        { Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Matrix Multiplication");

        job.setJarByClass(MatrixMultiplication.class);

        job.setMapperClass(MatrixMapper.class);

        job.setReducerClass(MatrixReducer.class);

        job.setOutputKeyClass(Text.class);

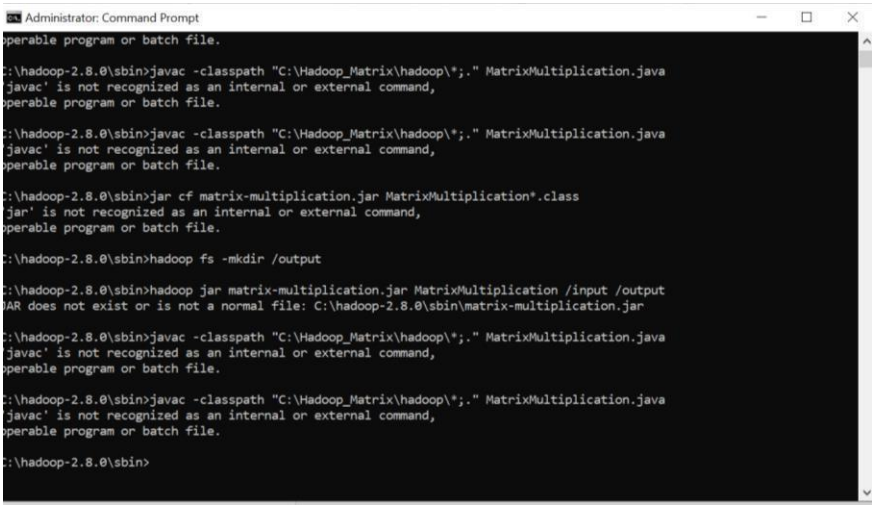
        job.setOutputValueClass(Text.class);

                FileInputFormat.addInputPath(job, new Path(args[0]));

        FileOutputFormat.setOutputPath(job, new Path(args[1]));

                System.exit(job.waitForCompletion(true) ? 0 : 1);

    }
```




```
}
```

RESULT:

EXP NO: 4	Run a basic Word Count Map Reduce program to understand Map Reduce Paradigm
Date:	

Aim :

To Run a basic Word Count Map Reduce program to understand Map Reduce Paradigm in Hadoop.

Procedure:

1. Download MapReduceClient.jar

(Link: https://github.com/MuhammadBilalYar/HADOOPINSTALLATION-ON-WINDOW-10/blob/master/MapReduceClient.jar)

2. Download Input_file.txt

(Link: https://github.com/MuhammadBilalYar/HADOOPINSTALLATION-ON-WINDOW-10/blob/master/input_file.txt)

Place both files in "C:/"

WordCount.java

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.Mapper;

import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
public static void main(String [] args) throws Exception
```

```
{
Configuration configuration =new Configuration();
Path inputFolder = new Path(args[0]);
Path outputFolder = new Path(args[1]);
Job job =new Job(configuration,"wordcount");
job.setJarByClass(WordCount.class);
job.setMapperClass(WordCountMapper.class);
job.setReducerClass(WordCountReducer.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job, inputFolder);
FileOutputFormat.setOutputPath(job, outputFolder);
System.exit(job.waitForCompletion(true)?0:1);
}
public static class WordCountMapper extends Mapper<LongWritable, Text, Text,
IntWritable>{ public void map(LongWritable key, Text value, Context con) throws IOException,
InterruptedException
{
String line = value.toString();
String[] words=line.split(" ");
for(String word: words )
{
Text outputKey = new Text(word.toUpperCase().trim());
IntWritable outputValue = new IntWritable(1);
con.write(outputKey, outputValue);
```

```
}
}
}
public static class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
public void reduce(Text word, Iterable<IntWritable> values, Context con) throws IOException,
InterruptedException
{
int sum = 0;
for(IntWritable value : values)
{
sum += value.get();
}
con.write(word, new IntWritable(sum));
}
}
}
```

To Do(Hadoop Operation)

Open cmd in Administrative mode and move to "C:/Hadoop-2.8.0/sbin" and start cluster

Hadoop Operation:

Start-all.cmd

1. Create an input directory in HDFS.
   hadoop fs -mkdir /ipdir

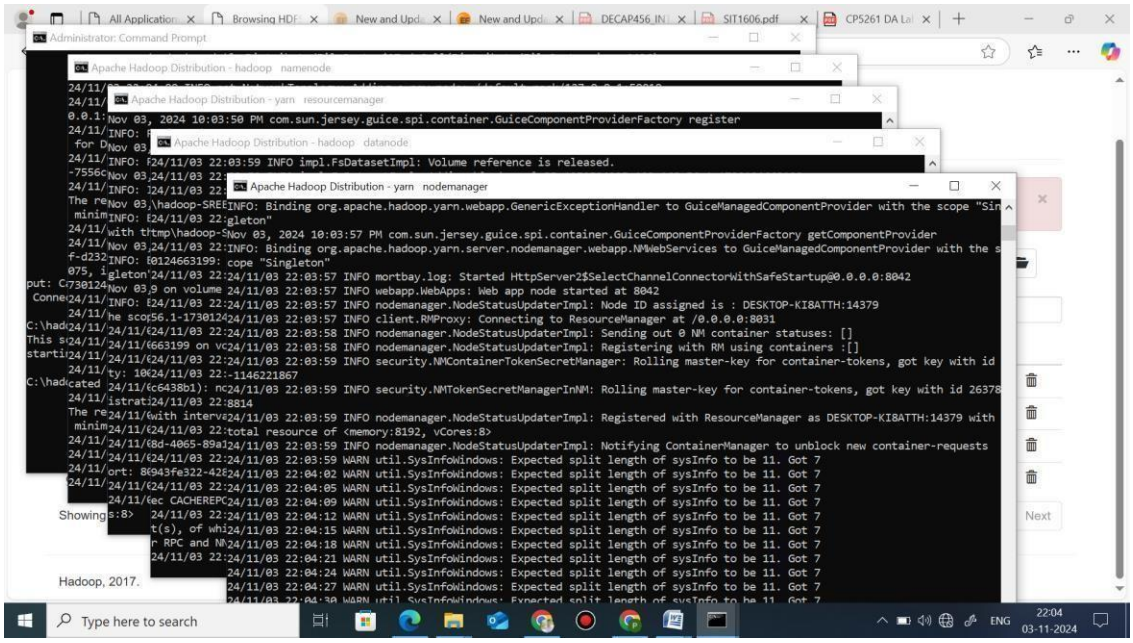
2. Copy the input text file named input_file.txt in the input directory(ipdir)

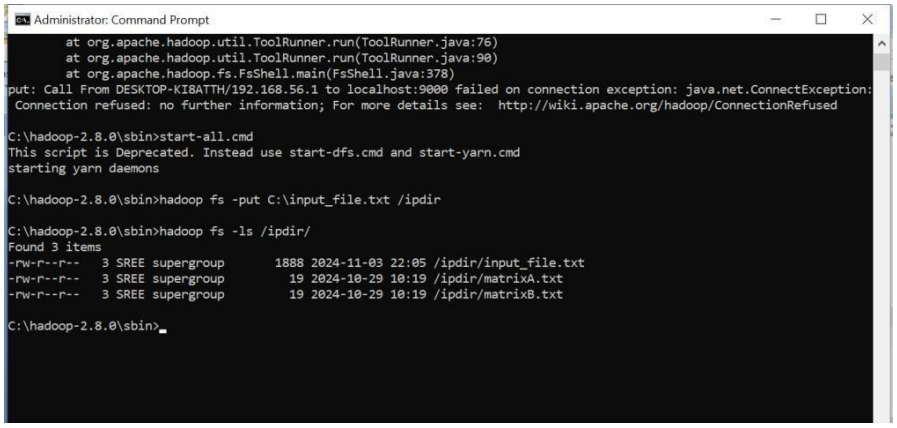
of HDFS.

hadoop fs -put C:/input_file.txt /ipdir

3. Verify input_file.txt available in HDFS input directory (ipdir).

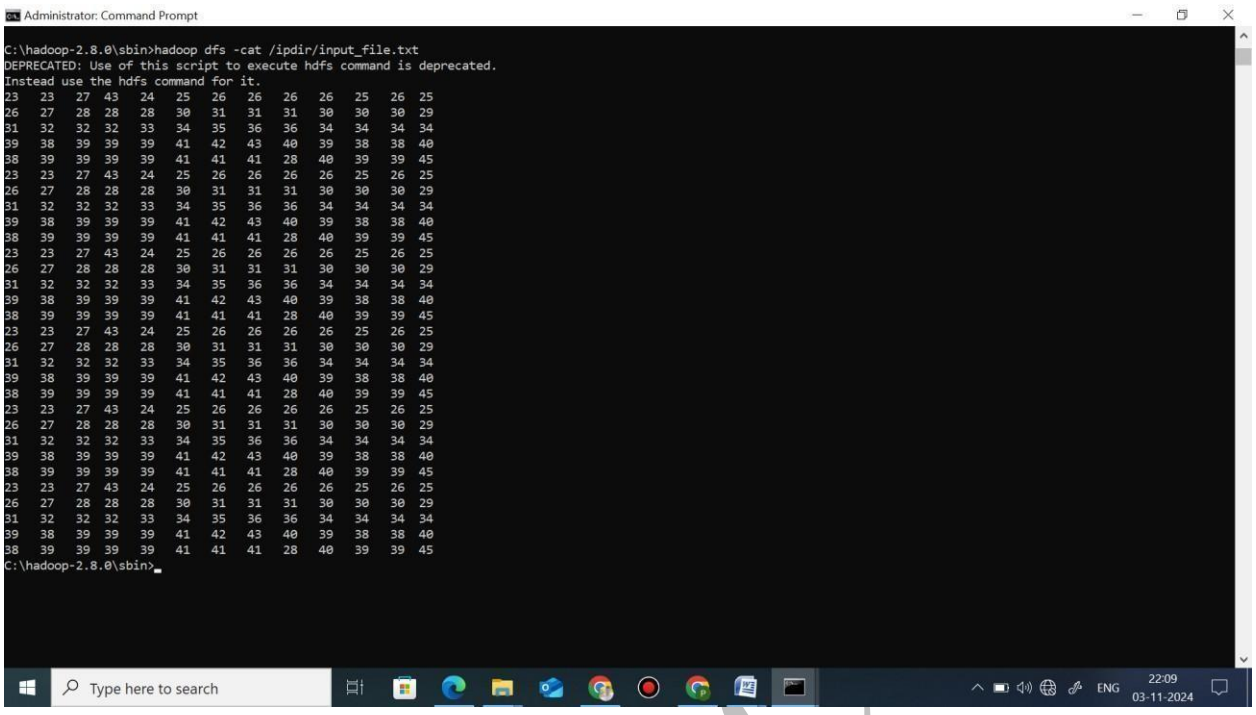
hadoop fs -ls /ipdir/





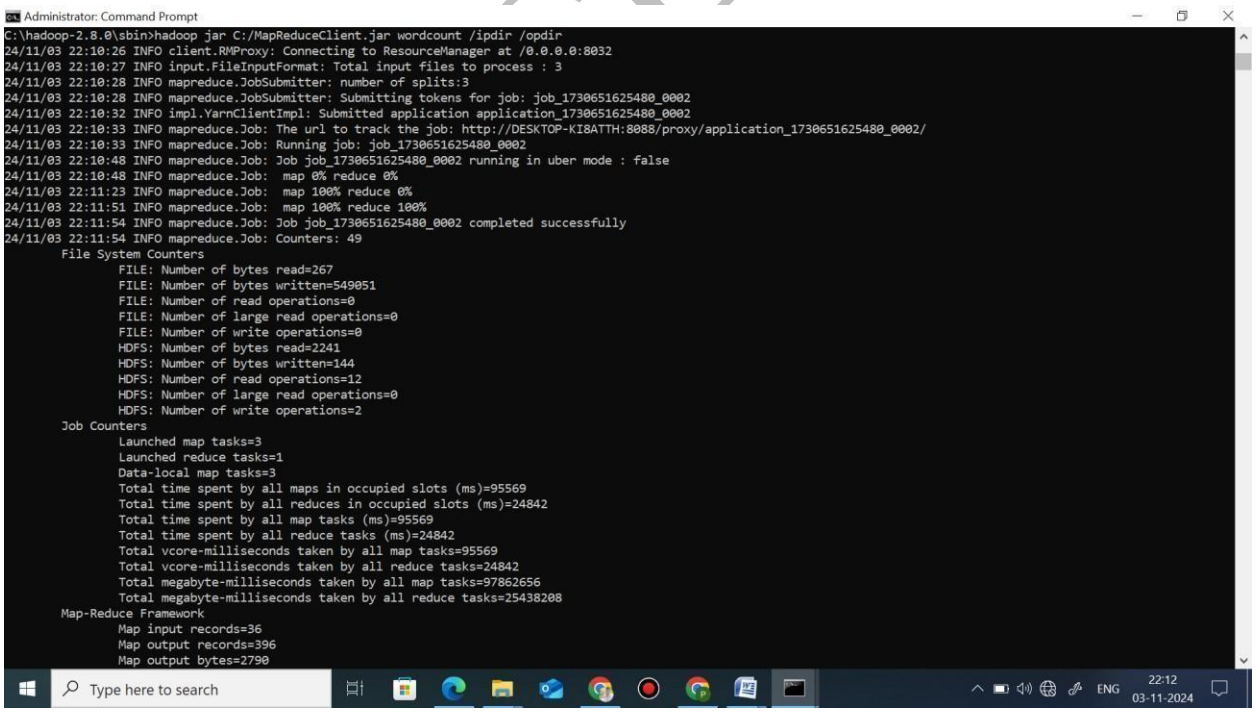
4. Verify content of the copied file.

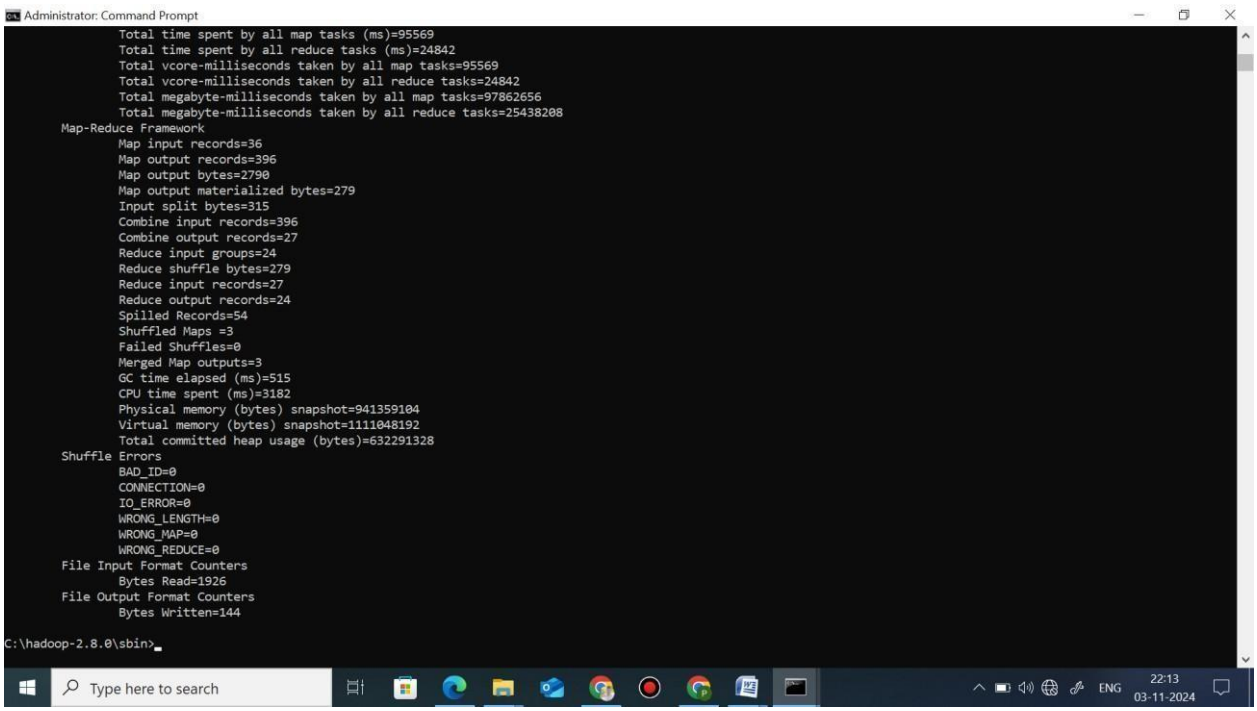
hadoop dfs -cat /ipdir/input_file.txt



5. Run MapReduceClient.jar and also provide input and out directories.

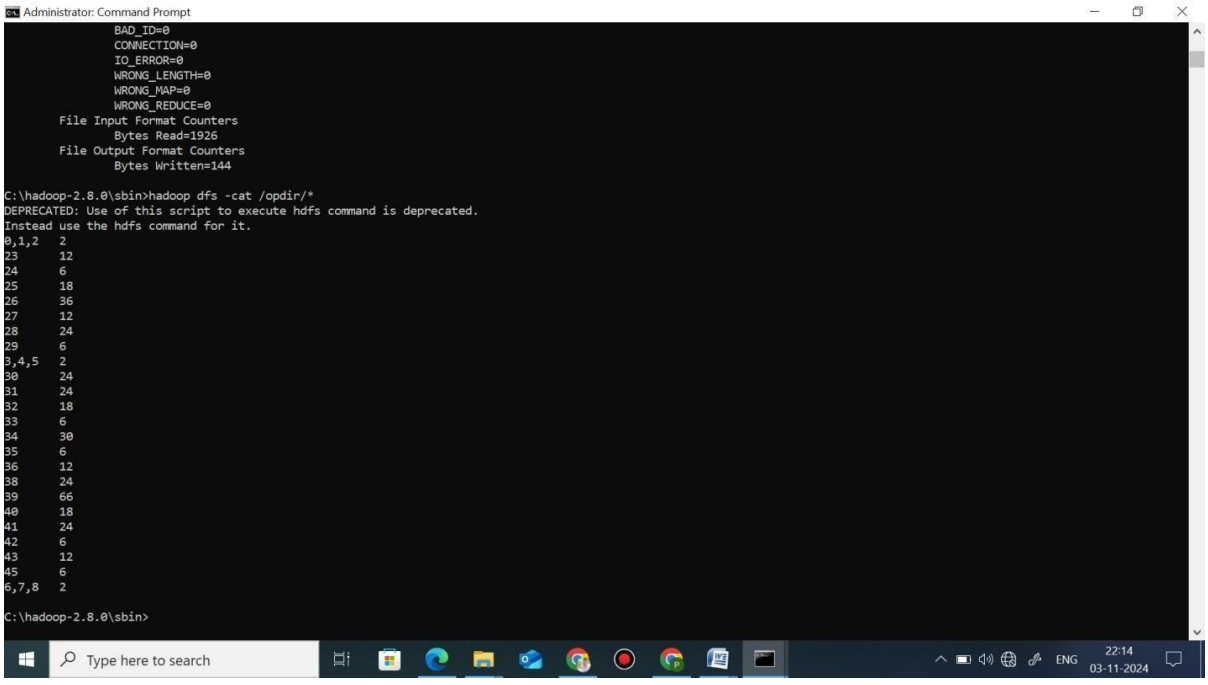
hadoop jar C:/MapReduceClient.jar wordcount /ipdir /opdir





6. Verify content for generated output file.

hadoop dfs -cat /opdir/*



Some Other useful commands

7) To leave Safe mode

hadoop dfsadmin –safemode leave

8) To delete file from HDFS directory

hadoop fs -rm -r /ipdir/input_file.txt

10) To delete directory from HDFS directory

hadoop fs -rm -r /ipdir

Some Other useful commands

8) To leave Safe mode

hadoop dfsadmin –safemode leave

hadoop dfs -cat /opdir/*

RESULT:

EXP NO: 5	Installation of Hive along with practice examples.
Date:	

Aim :

To install a hive with some examples.

Procedure:

Step 1: Start the Hive Shell

Open Command Prompt: Open your terminal or command prompt where Hadoop and Hive are installed.

hive

Step 2: Create a Database

Create a Database: Inside the Hive shell, run the following command to create a new database:

CREATE DATABASE test_db;

Use the Database: Switch to the newly created database:

USE test_db;

Step 3: Create a Table

Create a Table: Define a new table structure:

```
CREATE TABLE employees (
    id INT,
    name STRING,
    age INT,
    salary FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
```

STORED AS TEXTFILE;

Step 4: Load Data into the Table

Create a CSV File: Before loading data, make sure you have a CSV file ready. You can create it from your command prompt or terminal:

echo -e "1,John Doe,30,50000\n2,Jane Smith,25,60000\n3,Bob Johnson,40,70000" > employees.csv

Move the File to HDFS: Upload the CSV file to HDFS:

hadoop fs -mkdir /user/hive/warehouse/test_db.db

hadoop fs -put employees.csv /user/hive/warehouse/test_db.db/

Load Data into Hive Table: Back in the Hive shell, run:

LOAD DATA INPATH '/user/hive/warehouse/test_db.db/employees.csv' INTO TABLE employees;

Step 5: Query the Data

Select All Records: To see the data in the table, execute:

SELECT * FROM employees;

Expected Output:

OK

1 John Doe 30 50000.0

2 Jane Smith 25 60000.0

3 Bob Johnson 40 70000.0

Count the Number of Employees: To count how many employees are in the table:

SELECT COUNT(*) FROM employees;

Calculate Average Salary: To find the average salary:

SELECT AVG(salary) FROM employees;

Step 6: Exit the Hive Shell

After running your queries, you can exit the Hive shell by typing:

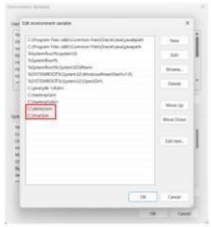
EXIT;

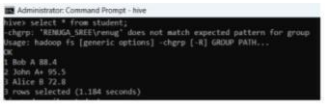


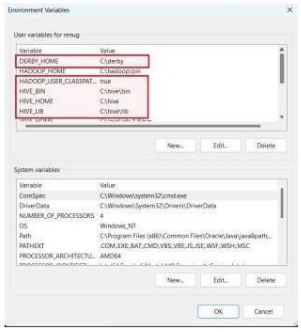


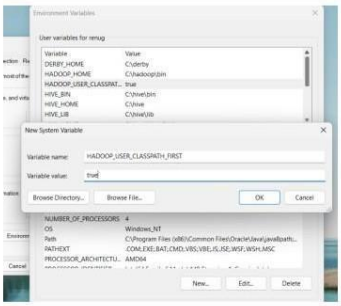












RESULT:

EXP NO: 6	Installation of HBase, Installing thrift along with Practice examples
Date:	

Aim :

To install HBase, Installing thrift along with Practice examples.

Procedure:

Java Installation: Make sure you have Java 8 or later installed on your system. You can check by running:

java -version

Hadoop Installation: HBase runs on top of Hadoop, so ensure Hadoop is installed and configured properly.

Maven: This is required for building HBase. You can install Maven if it's not already installed.

Environment Variables: Make sure the following environment variables are set:

JAVA_HOME: Path to your Java installation.

HADOOP_HOME: Path to your Hadoop installation.

Step 1: Download HBase

Go to the Apache HBase Releases page.

Download the latest stable release (e.g., hbase-2.x.x-bin.tar.gz).

Step 2: Extract HBase

Extract the downloaded file to your preferred directory. For example:

tar -xzf hbase-2.x.x-bin.tar.gz

mv hbase-2.x.x /usr/local/hbase

Step 3: Configure HBase

cd /usr/local/hbase/conf

Create a configuration file named hbase-site.xml:

<configuration>

<property>

```
    <name>hbase.rootdir</name>
    <value>hdfs://localhost:9000/hbase</value>
  </property>
  <property>
    <name>hbase.zookeeper.quorum</name>
    <value>localhost</value>
  </property>
</configuration>
```

Make sure you have the following configurations in hbase-env.sh:

export JAVA_HOME=/path/to/your/java

export HADOOP_HOME=/path/to/your/hadoop

Step 4: Start HBase

Open a terminal and start HBase using the following command:

/usr/local/hbase/bin/start-hbase.sh

Check if HBase is running by accessing the HBase Master UI:

Open your web browser and go to http://localhost:16010.

Step 5: Installing Thrift

HBase provides Thrift support for language-agnostic access to HBase tables.

Install Thrift: You need to install Thrift on your system. You can follow these steps depending on your operating system.

cd /usr/local/hbase

bin/hbase thrift start

Check Thrift Server: Verify that the Thrift server is running on port 9090.

Step 6: Practice Examples

Once HBase and Thrift are set up, you can run some basic commands to interact with HBase.

HBase Shell: Start the HBase shell:

/usr/local/hbase/bin/hbase shell

Create a Table:

create 'my_table', 'my_column_family'

Insert Data:

put 'my_table', 'row1', 'my_column_family:column1', 'value1'

Retrieve Data:

get 'my_table', 'row1'

Scan Table:

scan 'my_table'

Step 7: Using Thrift with HBase

You can interact with HBase using Thrift client. Here's an example in Python.

Install Thrift Python Client:

pip install thrift

Create a Simple Python Script:

```
from thrift import Thrift
from thrift.transport import TSocket
from thrift.transport import TTransport
from thrift.protocol import TBinaryProtocol
from hbase import Hbase

# Set up Thrift connection
```

```
transport = TSocket.TSocket('localhost', 9090)

transport = TTransport.TBufferedTransport(transport)

protocol = TBinaryProtocol.TBinaryProtocol(transport)

client = Hbase.Client(protocol)

transport.open()


# Example: list tables

tables = client.getTableNames()

print(tables)


transport.close()
```

Run Your Script: Save your script as hbase_thrift_example.py and run it:

python hbase_thrift_example.py

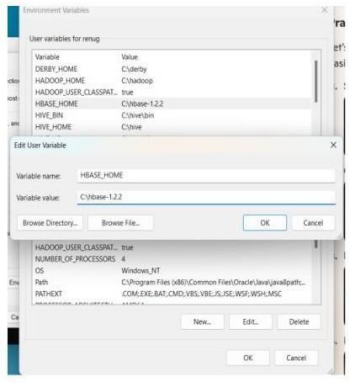
Step 8: Additional Practice

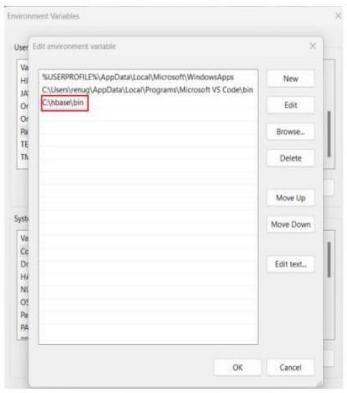
Try out the following operations using HBase shell and Thrift:

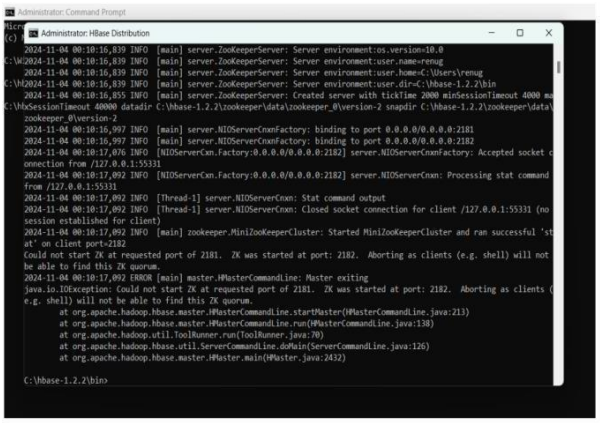
Updating records.

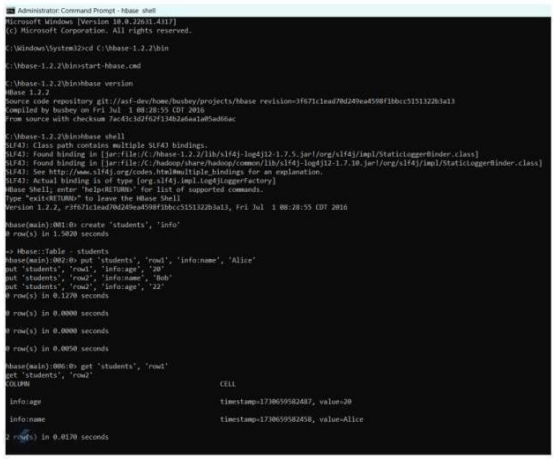
Deleting records.

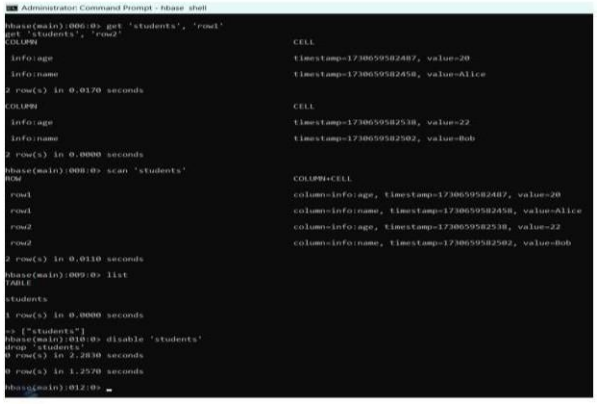
Creating more complex table schemas with multiple column families.






Administrator: Command Prompt
Administrator: HBase Distribution
2024-11-04 00:10:16,839 INFO [main] server.ZooKeeperServer: Server environment:os.version=10.0
2024-11-04 00:10:16,839 INFO [main] server.ZooKeeperServer: Server environment:user.name=renug
2024-11-04 00:10:16,839 INFO [main] server.ZooKeeperServer: Server environment:user.home=C:\Users\renug
2024-11-04 00:10:16,839 INFO [main] server.ZooKeeperServer: Server environment:user.dir=C:\hbase-1.2.2\bin
2024-11-04 00:10:16,855 INFO [main] server.ZooKeeperServer: Created server with tickTime 2000 minSessionTimeout 4000 ma
C:\hbase-1.2.2\zookeeper\data\zookeeper_0\version-2 snapdir C:\hbase-1.2.2\zookeeper\data\
zookeeper_0\version-2
2024-11-04 00:10:16,997 INFO [main] server.NIOServerCnxnFactory: binding to port 0.0.0.0/0.0.0.0:2181
2024-11-04 00:10:16,997 INFO [main] server.NIOServerCnxnFactory: binding to port 0.0.0.0/0.0.0.0:2182
2024-11-04 00:10:17,076 INFO [NIOServerCxn.Factory:0.0.0.0/0.0.0.0:2182] server.NIOServerCnxnFactory: Accepted socket c
onnection from /127.0.0.1:55331
2024-11-04 00:10:17,092 INFO [NIOServerCxn.Factory:0.0.0.0/0.0.0.0:2182] server.NIOServerCnxn: Processing stat command
from /127.0.0.1:55331
2024-11-04 00:10:17,092 INFO [Thread-1] server.NIOServerCnxn: Stat command output
2024-11-04 00:10:17,092 INFO [Thread-1] server.NIOServerCnxn: Closed socket connection for client /127.0.0.1:55331 (no
session established for client)
2024-11-04 00:10:17,092 INFO [main] zookeeper.MiniZooKeeperCluster: Started MiniZooKeeperCluster and ran successful 'st
at' on client port=2182
Could not start ZK at requested port of 2181. ZK was started at port: 2182. Aborting as clients (e.g. shell) will not
be able to find this ZK quorum.
2024-11-04 00:10:17,092 ERROR [main] master.HMasterCommandLine: Master exiting
java.io.IOException: Could not start ZK at requested port of 2181. ZK was started at port: 2182. Aborting as clients (
e.g. shell) will not be able to find this ZK quorum.
at org.apache.hadoop.hbase.master.HMasterCommandLine.startMaster(HMasterCommandLine.java:213)
at org.apache.hadoop.hbase.master.HMasterCommandLine.run(HMasterCommandLine.java:138)
at org.apache.hadoop.util.ToolRunner.run(ToolRunner.java:70)
at org.apache.hadoop.hbase.util.ServerCommandLine.doMain(ServerCommandLine.java:126)
at org.apache.hadoop.hbase.master.HMaster.main(HMaster.java:2432)
C:\hbase-1.2.2\bin>


Administrator: Command Prompt - hbase shell
Microsoft Windows [Version 10.0.22631.4317]
(c) Microsoft Corporation. All rights reserved.
C:\Windows\System32>cd C:\hbase-1.2.2\bin
C:\hbase-1.2.2\bin>start-hbase.cmd
C:\hbase-1.2.2\bin>hbase version
HBase 1.2.2
Source code repository git://asf-dev/home/busbey/projects/hbase revision=3f671c1ead70d249ea4598f1bbcc5151322b3a13
Compiled by busbey on Fri Jul 1 08:28:55 CDT 2016
From source with checksum 7ac43c3d2f62f134b2a6ea1a05ad66ac
C:\hbase-1.2.2\bin>hbase shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/C:/hbase-1.2.2/lib/slf4j-log4j12-1.7.5.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/C:/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.2.2, r3f671c1ead70d249ea4598f1bbcc5151322b3a13, Fri Jul 1 08:28:55 CDT 2016
hbase(main):001:0> create 'students', 'info'
0 row(s) in 1.5020 seconds
=> Hbase::Table - students
hbase(main):002:0> put 'students', 'row1', 'info:name', 'Alice'
put 'students', 'row1', 'info:age', '20'
put 'students', 'row2', 'info:name', 'Bob'
put 'students', 'row2', 'info:age', '22'
0 row(s) in 0.1270 seconds
0 row(s) in 0.0080 seconds
0 row(s) in 0.0080 seconds
0 row(s) in 0.0050 seconds
hbase(main):006:0> get 'students', 'row1'
get 'students', 'row2'
COLUMN CELL
info:age timestamp=1730659582487, value=20
info:name timestamp=1730659582458, value=Alice
2 row(s) in 0.0170 seconds


Administrator: Command Prompt - hbase shell
hbase(main):006:0> get 'students', 'row1'
get 'students', 'row2'
COLUMN CELL
info:age timestamp=1730659582487, value=20
info:name timestamp=1730659582458, value=Alice
2 row(s) in 0.0170 seconds
COLUMN CELL
info:age timestamp=1730659582538, value=22
info:name timestamp=1730659582502, value=Bob
2 row(s) in 0.0080 seconds
hbase(main):008:0> scan 'students'
ROW COLUMN+CELL
row1 column=info:age, timestamp=1730659582487, value=20
row1 column=info:name, timestamp=1730659582458, value=Alice
row2 column=info:age, timestamp=1730659582538, value=22
row2 column=info:name, timestamp=1730659582502, value=Bob
2 row(s) in 0.0110 seconds
hbase(main):009:0> list
TABLE
students
1 row(s) in 0.0080 seconds
=> ["students"]
hbase(main):010:0> disable 'students'
drop 'students'
0 row(s) in 2.2830 seconds
0 row(s) in 1.2570 seconds
hbase(main):012:0>

RESULT:

EXP NO: 7	Practice importing and exporting data from various databases.
Date:	

Aim :

To practice importing and exporting data from various databases.

Algorithm

Step 1: Import Data from the Database to Hadoop

Identify the source database (e.g., HBase, PostgreSQL).

Define connection parameters: database URL, username, and password.

Use a tool like Sqoop or Spark to read data from the database.

Store the imported data in Hadoop's HDFS or a Hive table.

Step 2: Process the Data

Use Hadoop processing frameworks like MapReduce, Hive, or Spark to perform transformations or analysis on the imported data.

Save the processed data back into HDFS or a Hive table.

Step 3: Export Data from Hadoop to the Database

Define the target database and table structure.

Use Sqoop or Spark to export the processed data from HDFS or Hive to the database.

Verify the exported data in the database.

Procedure

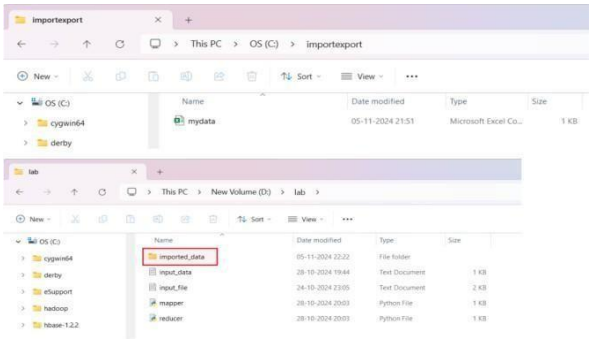
1. Tools and Environment Setup

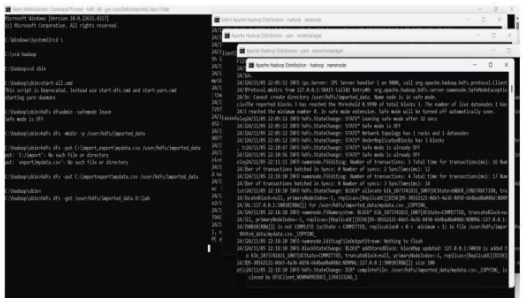
Install Hadoop and configure HDFS.

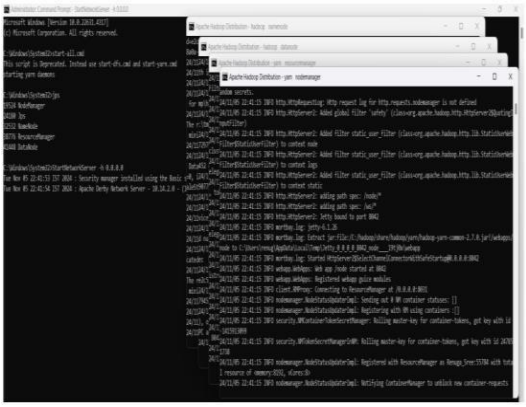
Install Sqoop for database integration.

Ensure the database (e.g., MySQL, PostgreSQL) is accessible from the Hadoop environment.

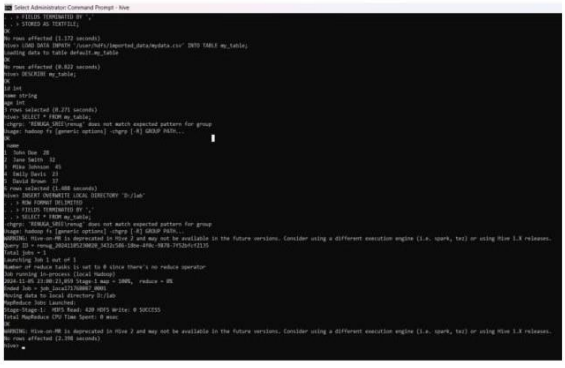
Install and configure Hive or Spark (if needed).











Result: